

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

HỌC PHẦN PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH



BÁO CÁO ASSIGNMENT 03

**ĐỀ TÀI: NEURAL NETWORKS AND
REPRESENTATION LEARNING**

Lớp:	D23CTPM01
Họ và tên:	Nguyễn Duy Nghĩa
Mã sinh viên:	B23DCCN600
Giảng viên hướng dẫn:	PGS.TS Trần Đình Quế
Học kỳ:	Học kỳ 1 năm học 2026 – 2027

Hà Nội, Tháng 09/2026

Mục lục

Chương 1. Cơ sở lý thuyết về mạng nơ-ron và học biểu diễn đặc trưng	1
1.1. Từ học máy truyền thống đến học biểu diễn sâu	1
1.2. Bản chất toán học: mạng nơ-ron là phép hợp hàm tham số hoá	2
1.3. Chứng minh toán học: vì sao bắt buộc phải có hàm kích hoạt phi tuyến?	2
1.4. Hàm kích hoạt ReLU và khởi tạo trọng số He Normal	3
1.5. Lan truyền tiến và lan truyền ngược thuần NumPy	4
1.5.1. Lan truyền tiến (Forward Pass)	4
1.5.2. Lan truyền ngược (Backward Pass)	4
1.6. Mười sáu khái niệm nền tảng trong mạng nơ-ron và học biểu diễn	5
Chương 2. Thực nghiệm cơ sở và phân tích bóc tách kiến trúc	8
2.1. Mô hình cơ sở và bốn khuyết tật phương pháp luận cố ý	8
2.1.1. Kết quả định lượng của mô hình cơ sở	10
2.2. Quy trình cải tiến chuẩn hoá y sinh	10
2.2.1. Làm sạch dữ liệu lâm sàng bằng trung vị	11
2.2.2. Triệt tiêu hoàn toàn rò rỉ dữ liệu	11
2.2.3. Phân chia phân tầng và dò tìm ngưỡng quyết định tối ưu	11
2.3. Bảng đối chứng trực diện: Cơ sở so với Cải tiến	13
2.4. Sáu nghiên cứu bóc tách kiến trúc (Ablation Studies)	13
2.4.1. Phân tích chi tiết từng nghiên cứu bóc tách	15
2.5. Minh chứng hình học về cơ chế học biểu diễn qua PCA 2D	15
2.6. Tổng kết bài học thực nghiệm nền tảng	16
Chương 3. Hệ thống 1 — Sàng lọc nguy cơ tiểu đường quy mô lớn (100,000 mẫu dữ liệu)	17
3.1. Bối cảnh y tế lâm sàng và khám phá dữ liệu (EDA)	17
3.2. Kiểm toán làm sạch dữ liệu và kỹ nghệ đặc trưng lâm sàng	21
3.2.1. Kỹ nghệ đặc trưng tương tác lâm sàng	21
3.2.2. Kiến trúc tiền xử lý ColumnTransformer chống rò rỉ dữ liệu	22
3.3. Khảo sát đánh đổi kiến trúc: Chiều sâu và Chiều rộng	22
3.4. Tinh chỉnh ngưỡng quyết định và giảm thiểu bỏ sót ca bệnh	24
3.5. Trực quan hoá quá trình học biểu diễn qua các tầng ẩn bằng PCA 2D	25
3.6. Tổng hợp đối chuẩn 4 mô hình (trước và sau tinh chỉnh ngưỡng)	26
3.6.1. Nhận xét chuyên sâu và đánh giá kết quả thực nghiệm y tế	27

Chương 4. Hệ thống 2 — Định giá bất động sản thực nghiệm (150,000 mẫu dữ liệu)	29
4.1. Bối cảnh thị trường bất động sản và phân tích EDA	29
4.2. Kiểm toán làm sạch dữ liệu và kỹ nghệ đặc trưng vị trí địa lý	31
4.2.1. Kỹ nghệ đặc trưng hình học và tỷ lệ không gian	32
4.2.2. Mã hoá đích Bayes làm mượt (Smooth Bayesian Target Encoding) chống rò rỉ	33
4.3. Khảo sát kiến trúc mạng hồi quy sâu: đột phá của chiều rộng	34
4.4. Phân tích chẩn đoán phần dư và sai số thị trường	35
4.5. Tổng hợp đối chuẩn 4 mô hình định giá bất động sản	36
4.5.1. Nhận xét chuyên sâu và đánh giá kết quả thực nghiệm định giá nhà	37
Chương 5. Hệ thống 3 — Phân loại nhận xét thương mại điện tử (22,636 đánh giá NLP)	39
5.1. Bối cảnh văn bản e-commerce và không gian TF-IDF	39
5.1.1. Quy trình làm sạch dữ liệu văn bản (NLP Text Cleaning)	41
5.1.2. Phân chia phân tầng và trích xuất TF-IDF chống rò rỉ	42
5.2. Khảo sát kiến trúc NLP: thích ứng trên dữ liệu thưa chiều cao	42
5.3. Tinh chỉnh ngưỡng cân bằng lớp (Macro F1 Optimization)	44
5.4. Trực quan hoá quá trình học biểu diễn ngữ nghĩa bằng PCA 2D	45
5.5. Tổng hợp đối chuẩn 4 mô hình xử lý văn bản NLP	46
5.5.1. Nhận xét chuyên sâu và đánh giá kết quả phân loại văn bản NLP	47
Chương 6. Tổng hợp đối chuẩn khoa học đa chiều và thảo luận chuyên sâu	49
6.1. Bảng tổng hợp đối chuẩn 12 mô hình trên cả ba hệ thống	49
6.2. Thảo luận về kiến trúc và triết lý lựa chọn mô hình	50
6.2.1. Không tồn tại một kiến trúc tối ưu cho mọi bài toán	50
6.2.2. Khi nào nên chọn Machine Learning, khi nào nên chọn Deep Learning?	51
6.2.3. Biểu diễn quan trọng hơn thuật toán	52
6.2.4. Tinh chỉnh ngưỡng — bước rẻ nhất nhưng bị bỏ quên nhiều nhất	52
Chương 7. Triển khai hệ thống thông minh: REST API, Web và Mobile	53
7.1. Kiến trúc triển khai ba dịch vụ độc lập	53
7.2. Tầng tri thức Neo4j — bổ sung tư vấn, không quyết định dự đoán	54
7.3. Giao diện Web	55
7.4. Giao diện Mobile	57
7.5. Hướng dẫn chạy lại toàn bộ hệ thống	61
Chương 8. Kết luận và kiểm toán khả năng tái lập	62

8.1. Tổng kết các đóng góp khoa học chính	62
8.2. Kiểm toán khả năng tái lập (Reproducibility Audit)	63
8.3. Hạn chế của nghiên cứu	63
8.4. Hướng phát triển và mở rộng nghiên cứu tiếp theo	64
8.5. Kho mã nguồn và tài nguyên đính kèm	65

Tài nguyên và mã nguồn dự án

Toàn bộ mã nguồn, dữ liệu, notebook thực nghiệm và báo cáo này được quản lý phiên bản công khai.

Kho lưu trữ mã nguồn

Hạng mục	Đường dẫn
Kho lưu trữ GitHub của dự án	https://github.com/Nghaiz/Assignment-03-Intelligent-System
Báo cáo nghiên cứu (tệp này)	report/A03_CT_nghiand.600.pdf
Hướng dẫn cài đặt và chạy lại	README.md
Danh sách phụ thuộc	requirements.txt

Bốn bộ dữ liệu thực nghiệm

Sử dụng ở	Bộ dữ liệu	Nguồn Kaggle
Chương 2 — Mô hình cơ sở & Bóc tách	Pima Indians Diabetes Database (768 bản ghi)	https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database
Chương 3 — Hệ thống 1	Diabetes Prediction Dataset (100,000 bản ghi)	https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset
Chương 4 — Hệ thống 2	USA Real Estate Dataset (150,000 bản ghi)	https://www.kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset
Chương 5 — Hệ thống 3	Women's Clothing E-Commerce Reviews (23,486 bản ghi)	https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

Tổng cộng **274,254 bản ghi** dữ liệu thực tế.

Năm notebook thực nghiệm

Notebook	Đường dẫn trong kho mã nguồn
Notebook 01 — Mạng cơ sở viết tay (768 mẫu Pima)	<code>diabetes_baseline/notebook/01_diabetes_dl_from_scratch_numpy.ipynb</code>
Notebook 02 — Cải tiến mô hình & 6 nghiên cứu bóc tách	<code>diabetes_baseline/notebook/02_diabetes_dl_from_scratch_improvements.ipynb</code>
Notebook 03 — Hệ thống 1: Sàng lọc tiểu đường 100,000 mẫu	<code>diabetes_large/notebook/03_diabetes_large_ml_vs_dl_scratch.ipynb</code>
Notebook 04 — Hệ thống 2: Định giá bất động sản 150,000 mẫu	<code>house_price_large/notebook/04_house_price_large_ml_vs_dl_scratch.ipynb</code>
Notebook 05 — Hệ thống 3: Phân loại nhận xét NLP 23,486 mẫu	<code>customer_comments/notebook/05_comments_large_ml_vs_dl_scratch.ipynb</code>

Ba dịch vụ REST API kèm giao diện Web và Mobile

Dịch vụ	Cổng	Endpoint dự đoán
Hệ thống 1 — Sàng lọc tiểu đường	5001	POST /diabetes/v1/predict
Hệ thống 2 — Định giá bất động sản	5002	POST /house-price/v1/predict
Hệ thống 3 — Phân loại nhận xét	5003	POST /comments/v1/predict

Mã nguồn ba dịch vụ nằm ở `<tên_hệ_thống>/api/rest_api.py`. Mỗi dịch vụ tự phục vụ luôn giao diện Web tại `/` và giao diện Mobile tại `/mobile`; ba cổng khác nhau nên chạy đồng thời được.

Chương 1. Cơ sở lý thuyết về mạng nơ-ron và học biểu diễn đặc trưng

Báo cáo này trình bày một chuỗi thực nghiệm về mạng nơ-ron sâu, trong đó ràng buộc trung tâm là: **toàn bộ mô hình học sâu được hiện thực hoá 100% bằng NumPy**. Không TensorFlow, không PyTorch, không `sklearn.neural_network`. Mọi phép nhân ma trận, mọi đạo hàm riêng, mọi bước cập nhật trọng số đều được viết tay.

Ràng buộc này không nhằm gây khó. Nó phục vụ một mục đích cụ thể: khi tự tay lập trình lan truyền tiến và lan truyền ngược, ta buộc phải hiểu tường minh bản chất dòng chảy tensor, cơ chế cập nhật gradient, và vai trò của từng hàm toán học. Điều này loại bỏ hoàn toàn tư duy xem mạng nơ-ron là một chiếc hộp đen bí ẩn.

1.1. Từ học máy truyền thống đến học biểu diễn sâu

Trong quy trình học máy truyền thống, chu trình phát triển hệ thống thông minh thường tuân theo mô hình tuần tự:

$$X \rightarrow [\text{Kỹ nghệ đặc trưng thủ công}] \rightarrow X_{\text{engineered}} \rightarrow [\text{Thuật toán học máy}] \rightarrow \hat{y}$$

Hiệu năng của các mô hình tuyến tính, cây quyết định hay Naive Bayes phụ thuộc rất lớn vào chất lượng của các thuộc tính do chuyên gia con người trích xuất. Cách tiếp cận này bộc lộ giới hạn nghiêm trọng khi dữ liệu có cấu trúc phi tuyến cao, tương tác phức tạp, hoặc số chiều lớn.

Câu hỏi nghiên cứu trung tâm được đặt ra: *liệu mô hình có thể tự động học các không gian biểu diễn hữu ích trực tiếp từ dữ liệu hay không?*

$$X \rightarrow [\text{Biến đổi tham số hoá } h_{\theta}(X)] \rightarrow H \rightarrow [\text{Tầng dự đoán}] \rightarrow \hat{y}$$

Điểm cốt lõi là không gian ẩn H không do con người định nghĩa trước, mà các tham số $\theta = \{W, b\}$ của nó được tối ưu hoá *đồng thời* cùng hàm mục tiêu thông qua giải thuật lan truyền ngược.

1.2. Bản chất toán học: mạng nơ-ron là phép hợp hàm tham số hoá

Một mạng nơ-ron đa tầng (Multi-Layer Perceptron) không phải chiếc hộp ma thuật. Về bản chất nó là một hàm toán học tham số hoá được xây dựng từ phép hợp của nhiều phép biến đổi liên tiếp:

$$\hat{y} = f_{\theta}(x) = (f_L \circ f_{L-1} \circ \dots \circ f_2 \circ f_1)(x)$$

Trong đó mỗi tầng nơ-ron $\ell \in \{1, 2, \dots, L\}$ thực hiện đúng hai thao tác tuần tự:

1. **Biến đổi affine.** Chiếu vector đầu vào từ không gian $\mathbb{R}^{d_{\ell-1}}$ sang không gian $\mathbb{R}^{d_{\ell}}$:

$$Z^{[\ell]} = A^{[\ell-1]} W^{[\ell]} + b^{[\ell]}$$

với $W^{[\ell]} \in \mathbb{R}^{d_{\ell-1} \times d_{\ell}}$ là ma trận trọng số và $b^{[\ell]} \in \mathbb{R}^{1 \times d_{\ell}}$ là vector độ chệch.

2. **Kích hoạt phi tuyến.** Áp dụng hàm $\varphi(\cdot)$ trên từng phần tử để sinh ra biểu diễn ẩn:

$$A^{[\ell]} = \varphi(Z^{[\ell]})$$

Vai trò của ma trận trọng số W. Định hình độ dốc, co giãn và xoay không gian đặc trưng — xác định mức độ ảnh hưởng của từng nơ-ron tầng trước lên tầng sau.

Vai trò của vector độ chệch b. Dịch chuyển siêu phẳng quyết định trong không gian mà không bị gò bó phải đi qua gốc tọa độ.

1.3. Chứng minh toán học: vì sao bắt buộc phải có hàm kích hoạt phi tuyến?

Đây là mệnh đề nền tảng của toàn bộ Deep Learning, và nó chứng minh được bằng vài dòng đại số. Giả sử ta loại bỏ hàm kích hoạt phi tuyến (tức chọn $\varphi(z) = z$ là hàm đồng nhất). Xét mạng hai tầng ẩn:

$$h_1 = xW_1 + b_1$$

$$h_2 = h_1W_2 + b_2 = (xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$$

Đặt $W_{\text{composite}} = W_1W_2$ và $b_{\text{composite}} = b_1W_2 + b_2$, ta thu được:

$$h_2 = xW_{\text{composite}} + b_{\text{composite}}$$

Kết luận toán học. Phép hợp của bất kỳ số lượng tầng tuyến tính nào cũng chỉ tương đương với **một** phép biến đổi affine duy nhất. Việc xếp chồng nhiều tầng tuyến tính hoàn toàn không làm tăng khả năng biểu diễn (Expressive Power) của mạng so với mô hình hồi quy tuyến tính ban đầu. Do đó, tính phi tuyến là điều kiện tiên quyết tạo nên sức mạnh của Deep Learning.

Mệnh đề này không dừng lại ở lý thuyết. Ở mục 2.3, tôi sẽ **kiểm chứng nó bằng thực nghiệm có kiểm soát**: huấn luyện một mạng y hệt nhưng bỏ hẵn hàm ReLU, rồi đo xem hiệu năng tụt bao nhiêu.

1.4. Hàm kích hoạt ReLU và khởi tạo trọng số He Normal

Trong toàn bộ các kiến trúc mạng nơ-ron sâu được xây dựng trong báo cáo, hàm kích hoạt **ReLU (Rectified Linear Unit)** được lựa chọn cho tất cả các tầng ẩn:

$$\text{ReLU}(z) = \max(0, z) \quad \cdot \quad d/dz \text{ReLU}(z) = 1 \text{ nếu } z > 0, \text{ và } = 0 \text{ nếu } z \leq 0$$

Hai ưu thế quyết định của ReLU:

- **Triệt tiêu hiện tượng bão hoà đạo hàm (Vanishing Gradient).** Khi $z > 0$, đạo hàm luôn bằng 1, cho phép gradient truyền ngược qua nhiều tầng mà không bị suy giảm theo cấp số nhân như Sigmoid hay Tanh.
- **Tạo tính thưa thớt (Sparsity).** Các nơ-ron có giá trị $z \leq 0$ bị khoá về đúng 0, giúp mạng học biểu diễn đặc trưng phân tách rời rạc và tăng tốc độ tính toán ma trận.

Khởi tạo trọng số He Normal (Kaiming Normal). Để ngăn hiện tượng nổ gradient (Exploding) hoặc biến mất gradient ngay từ epoch đầu tiên, trọng số của mỗi tầng được khởi tạo theo phân phối Gauss có phương sai chuẩn hoá theo số nút vào d_{in} :

$$W^{[\ell]} \sim \mathcal{N}(0, \sqrt{2 / d_{in}}) \quad \cdot \quad b^{[\ell]} = 0$$

Hệ số $\sqrt{2}$ được thiết kế riêng cho ReLU: vì ReLU khoá khoảng một nửa số nơ-ron về 0, phương sai tín hiệu bị giảm một nửa qua mỗi tầng; nhân bù hệ số 2 giữ cho phương sai ổn định xuyên suốt độ sâu của mạng.

1.5. Lan truyền tiến và lan truyền ngược thuần NumPy

Quá trình tối ưu hoá mạng nơ-ron được thực hiện thông qua Gradient Descent. Phần này ghi lại đầy đủ hệ phương trình mà toàn bộ mã nguồn trong năm notebook hiện thực hoá nguyên văn.

1.5.1. Lan truyền tiến (Forward Pass)

Với các tầng ẩn $\ell = 1, \dots, L-1$:

$$Z^{[\ell]} = A^{[\ell-1]}W^{[\ell]} + b^{[\ell]} \quad \cdot \quad A^{[\ell]} = \max(0, Z^{[\ell]})$$

Tại tầng ra L, hình thức phụ thuộc vào loại bài toán:

Bài toán phân loại nhị phân (Chương 2, 3 và 5):

$$\hat{y} = \sigma(Z^{[L]}) = 1 / (1 + e^{-Z}) \in (0, 1)$$

với hàm mất mát Binary Cross-Entropy trên batch gồm m mẫu:

$$\mathcal{L}_{BCE} = -(1/m) \sum [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Bài toán hồi quy (Chương 4) — tầng ra *tuyến tính*:

$$\hat{y} = Z^{[L]} \quad \cdot \quad \mathcal{L}_{MSE} = (1/2m) \sum (\hat{y}^{(i)} - y^{(i)})^2$$

1.5.2. Lan truyền ngược (Backward Pass)

Tại tầng ra L, gradient sai số được rút gọn kỳ diệu nhờ sự triệt tiêu giữa đạo hàm hàm mất mát và đạo hàm hàm kích hoạt:

$$dZ^{[L]} = (1/m)(\hat{y} - y)$$

Một điểm thú vị đáng ghi nhận. Công thức gradient tầng ra của *BCE + Sigmoid* và của *MSE + Linear* hoàn toàn giống nhau. Đây không phải trùng hợp ngẫu nhiên — cả hai cặp đều thuộc họ hàm mất mát chính tắc (canonical link) của phân phối mũ tương ứng. Hệ quả thực tiễn: toàn bộ phần lan truyền ngược qua các tầng ẩn giữ nguyên không đổi giữa bài toán phân loại và bài toán hồi quy, chỉ cần đổi hàm kích hoạt tầng cuối.

Gradient của trọng số và độ chệch tại tầng L:

$$dW^{[L]} = (A^{[L-1]})^T dZ^{[L]} \cdot db^{[L]} = \sum_i dZ^{[L]}$$

Truyền ngược gradient về các tầng ẩn $\ell = L-1, L-2, \dots, 1$:

$$dA^{[\ell]} = dZ^{[\ell+1]}(W^{[\ell+1]})^T \cdot dZ^{[\ell]} = dA^{[\ell]} \odot \mathbb{I}(Z^{[\ell]} > 0)$$

$$dW^{[\ell]} = (A^{[\ell-1]})^T dZ^{[\ell]} \cdot db^{[\ell]} = \sum_i dZ^{[\ell]}$$

Cập nhật trọng số theo luật Gradient Descent với tốc độ học η :

$$W^{[\ell]} \leftarrow W^{[\ell]} - \eta \cdot dW^{[\ell]} \cdot b^{[\ell]} \leftarrow b^{[\ell]} - \eta \cdot db^{[\ell]}$$

PYTHON

```
def _backward(self, y):
    """Lan truyền ngược – bản dịch trực tiếp hệ phương trình ở mục 1.5.2 sang
    code."""
    dZ = (self.As[-1] - y) / y.shape[0]          # dZ^[L] = (ŷ - y) / m
    for i in range(self.L - 1, -1, -1):
        dW = self.As[i].T @ dZ                    # dW^[ℓ]
        db = dZ.sum(axis=0, keepdims=True)         # db^[ℓ]
        if i > 0:
            dZ = (dZ @ self.W[i].T) * (self.Zs[i - 1] > 0) # dA ⊙ I(Z > 0)
            self.W[i] -= self.lr * dW              # cập nhật Gradient Descent
            self.b[i] -= self.lr * db
```

Toàn bộ thuật toán lan truyền ngược trong 9 dòng NumPy — dùng chung cho cả năm notebook.

1.6. Mười sáu khái niệm nền tảng trong mạng nơ-ron và học biểu diễn

Để củng cố cơ sở lý luận trước khi đi vào các bài toán thực nghiệm đối đầu, phần này tổng hợp có hệ thống 16 vấn đề khoa học cốt lõi định hình nguyên lý vận hành của mạng nơ-ron sâu.

Bảng 1. Mười sáu khái niệm nền tảng định hình nguyên lý vận hành của mạng nơ-ron sâu

#	Khái niệm	Nội dung cốt lõi
1	Khác biệt giữa ML truyền thống và Deep Learning	ML truyền thống đòi hỏi con người thiết kế đặc trưng thủ công rồi đưa vào thuật toán tuyến tính. Deep Learning học đồng thời cả không gian biểu diễn đặc trưng ẩn lẫn hàm phân loại/hồi quy.
2	Ý nghĩa của phương trình $\hat{y} = f_3(f_2(f_1(X)))$	Mạng nơ-ron là phép hợp của ba biến đổi liên tiếp: f_1 học biểu diễn sơ cấp, f_2 kết hợp thành biểu diễn trừu tượng, f_3 ánh xạ thành giá trị dự đoán.
3	Mục đích của ma trận trọng số W	Định hình độ dốc, co giãn và xoay không gian đặc trưng; xác định mức độ ảnh hưởng của từng nơ-ron tầng trước lên tầng sau.
4	Mục đích của vector độ lệch b	Dịch chuyển siêu phẳng quyết định trong không gian mà không bị gò bó phải đi qua gốc tọa độ.
5	Vì sao cần hàm kích hoạt phi tuyến	Không có phi tuyến, phép hợp của mọi tầng chỉ là một hàm tuyến tính đơn lẻ, làm mất toàn bộ khả năng xấp xỉ các quan hệ phức tạp (chứng minh ở mục 1.3).
6	Hàm ReLU thực hiện thao tác gì	Lọc bỏ toàn bộ giá trị âm (gán về 0) và giữ nguyên giá trị dương, đưa tính phi tuyến và tính thưa thớt vào mạng.
7	Vì sao dùng Sigmoid ở tầng cuối bài toán phân loại	Sigmoid nén giá trị số thực $(-\infty, +\infty)$ về khoảng xác suất $(0, 1)$, phù hợp với phân phối Bernoulli của biến cố nhị phân.
8	Vì sao cần hàm mất mát	Cung cấp thước đo định lượng về sai lệch giữa dự đoán và thực tế, đóng vai trò hàm mục tiêu để thuật toán tối ưu hoá dẫn dắt việc học.
9	Binary Cross-Entropy đo lường điều gì	Đo khoảng cách phân phối xác suất (KL divergence) giữa phân phối thực tế $y \in \{0, 1\}$ và phân phối dự báo $\hat{y} \in (0, 1)$.
10	Gradient là gì	Vector chứa các đạo hàm riêng của hàm mất mát đối với từng trọng số, chỉ hướng tăng nhanh nhất của hàm mất mát trong không gian tham số.
11	Thuật toán Backpropagation tính toán điều gì	Áp dụng quy tắc dây chuyền (Chain Rule) để tính chính xác gradient $\nabla_{\theta} \mathcal{L}$ từ tầng ra ngược về tầng vào một cách hiệu quả.

#	Khái niệm	Nội dung cốt lõi
1 2	Giải thuật Gradient Descent làm gì	Cập nhật trọng số theo hướng ngược chiều gradient ($-\nabla_{\theta} \mathcal{L}$) để từng bước tiến dần về cực tiểu của hàm mất mát.
1 3	Vì sao trọng số được cập nhật lặp lại nhiều lần (Epochs)	Vì bề mặt hàm mất mát là phi tuyến đa chiều, mỗi bước cập nhật chỉ di chuyển một bước nhỏ η ; cần nhiều bước để mạng hội tụ về trạng thái tối ưu.
1 4	Vì sao phải chia tách dữ liệu Train và Test	Để kiểm tra khả năng tổng quát hoá (Generalization) của mô hình trên dữ liệu chưa từng thấy, phát hiện và ngăn chặn hiện tượng học vẹt (Overfitting).
1 5	Vì sao phải chuẩn hoá đặc trưng	Giúp mặt phẳng mức hàm mất mát có dạng tròn đều đối xứng, giúp gradient descent hội tụ nhanh và ổn định, tránh dao động hình dích-dắc.
1 6	Khác biệt giữa xác suất và phân lớp dự đoán	Xác suất $\hat{y} \in [0, 1]$ biểu thị mức độ tự tin liên tục của mô hình; phân lớp dự đoán $\hat{y}_{\text{class}} \in \{0, 1\}$ là quyết định rời rạc thu được sau khi áp dụng ngưỡng quyết định (Threshold cut-off).

Chương 2. Thực nghiệm cơ sở và phân tích bóc tách kiến trúc

Bước đầu tiên của chuỗi nghiên cứu là xây dựng một mạng nơ-ron ba tầng thuần NumPy trên tập dữ liệu chuẩn đoán tiểu đường gốc *Pima Indians Diabetes* gồm 768 bệnh nhân với 8 đặc trưng y sinh.

Mục tiêu của chương này **không phải đạt điểm số cao**. Mục tiêu là dựng một mô hình cơ sở có khuyết tật cố ý, phơi bày trọn vẹn các lỗi phương pháp luận thường gặp, rồi sửa từng lỗi một và đo chính xác mức cải thiện thu được.

2.1. Mô hình cơ sở và bốn khuyết tật phương pháp luận cố ý

Bảng 2. Cấu hình kiến trúc và siêu tham số của mô hình cơ sở

Hạng mục cấu hình	Giá trị
Cấu hình mạng	8 -> 16 -> 8 -> 1 (2 tầng ẩn ReLU, tầng ra Sigmoid)
Số tham số học được	289 tham số
Khởi tạo trọng số	$N(0,1)$ standard normal
Tốc độ học η	0.01
Số chu kỳ huấn luyện	1,000 epochs
Phương pháp cập nhật	Full-Batch Gradient Descent
Ngưỡng quyết định	0.50 (cố định, cứng nhắc)

Bốn khuyết tật được cài vào một cách có chủ đích, mỗi khuyết tật đại diện cho một lỗi phương pháp luận rất phổ biến:

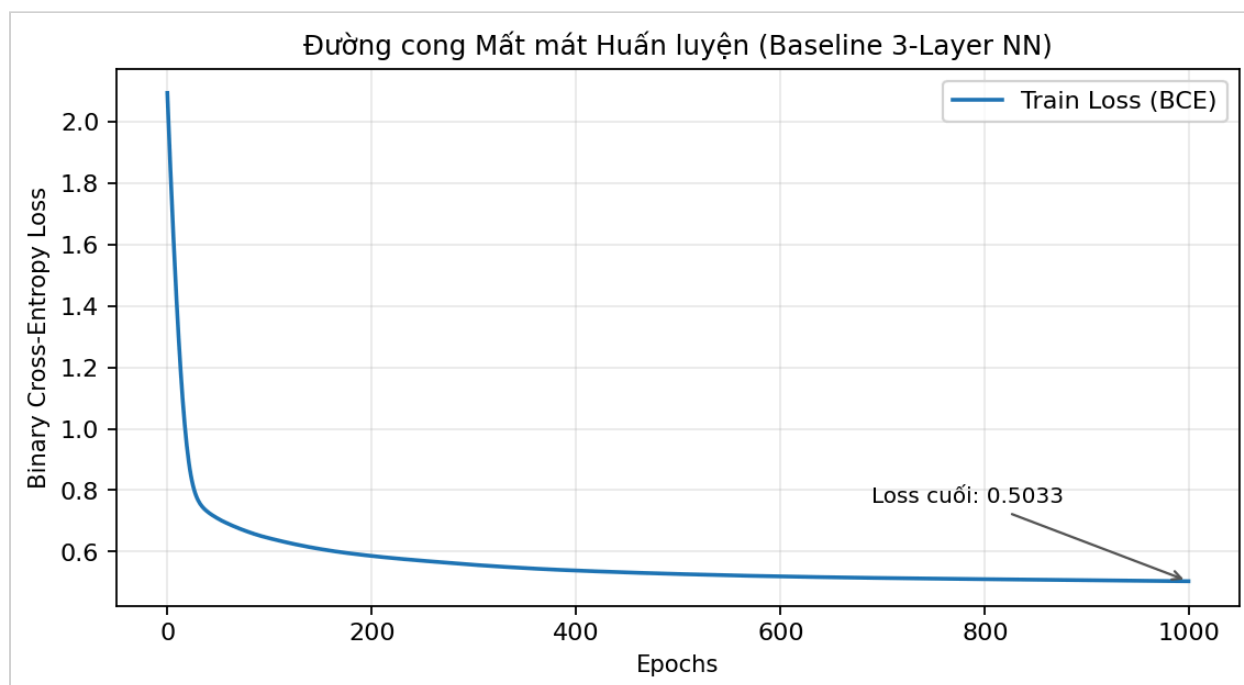
1. **Rò rỉ dữ liệu (Data Leakage)**. Phép chuẩn hoá Z-Score được áp dụng trên *toàn bộ* tập dữ liệu *trước khi* phân chia Train/Test. Vì μ_{global} và σ_{global} chứa thông tin của tập kiểm thử, mô hình đã 'nhìn trộm' phân phối dữ liệu mà lẽ ra nó chưa từng thấy. Mọi điểm số thu được đều lạc quan giả tạo và mất tính khách quan khoa học.
2. **Phân chia ngẫu nhiên không phân tầng**. Việc chia 80/20 bằng hoán vị ngẫu nhiên thuần làm sai lệch tỷ lệ bệnh nhân mắc bệnh giữa hai tập: tập Test chỉ có 33.1%

dương tính trong khi quần thể là 34.9%.

3. **Bỏ qua các giá trị 0 phi lý sinh học.** Trong y học, người sống không thể có nồng độ glucose huyết, huyết áp tâm trương hay chỉ số BMI bằng 0. Các số 0 này thực chất là giá trị thiếu bị mã hoá nhầm.
4. **Áp đặt ngưỡng quyết định cứng nhắc 0.50.** Khiến mô hình thiên lệch về nhãn đa số, dẫn đến tỷ lệ bỏ sót ca bệnh rất cao.

Bảng 3. Kiểm toán các giá trị 0 phi lý sinh học trong tập Pima gốc — mô hình cơ sở giữ nguyên toàn bộ

Chỉ số y sinh	Số bản ghi bằng 0	Tỷ lệ trên tổng mẫu
Insulin	374	48.70%
SkinThickness	227	29.56%
BloodPressure	35	4.56%
BMI	11	1.43%
Glucose	5	0.65%



Hình 1. Đường cong suy giảm hàm mất mát Binary Cross-Entropy của mô hình cơ sở

Quan sát. Hàm mất mát giảm đều và trơn từ 2.0942 xuống 0.5033 sau 1,000 epochs, không có dao động.

Diễn giải. Đường cong mượt chứng tỏ các phương trình lan truyền ngược viết tay hoạt động đúng về mặt toán học: gradient được tính chính xác và truyền qua ba tầng ổn định.

Ý nghĩa với học máy. Loss huấn luyện giảm **không** đồng nghĩa với năng lực sàng lọc lâm sàng. Một đường cong đẹp có thể che giấu một mô hình vô dụng trên tập kiểm thử — đó chính xác là điều xảy ra ở đây.

2.1.1. Kết quả định lượng của mô hình cơ sở

Bảng 4. Kết quả mô hình cơ sở trên 154 mẫu kiểm thử (ngưỡng cứng 0.50)

Chỉ số đánh giá	Giá trị	Diễn giải
Accuracy (Độ chính xác)	66.88 %	Trên tập kiểm thử độc lập
Precision (Độ chuẩn xác)	50.00 %	Trong số ca báo dương, bao nhiêu là đúng
Recall (Độ nhạy phát hiện bệnh)	39.22 %	Trong số ca bệnh thực tế, phát hiện được bao nhiêu
F1-Score	43.96 %	Trung bình điều hoà Precision và Recall
Số ca bệnh bị bỏ sót (FN)	31 ca	Trên tổng 51 ca dương tính thực tế

Vấn đề nghiêm trọng nhất. Recall chỉ đạt 39.22% — nghĩa là mô hình **bỏ sót 31 trong tổng số 51 bệnh nhân thực sự mắc bệnh**. Trong bối cảnh sàng lọc y tế, đây là một thất bại nghiêm trọng: mỗi ca bỏ sót là một bệnh nhân mất cơ hội can thiệp điều trị sớm.

2.2. Quy trình cải tiến chuẩn hoá y sinh

Nhằm khắc phục triệt để các hạn chế trên, tôi thiết lập quy trình thực nghiệm cải tiến với năm thay đổi. Điều quan trọng cần nhấn mạnh: **kiến trúc mạng được giữ nguyên hoàn toàn** (8 -> 16 -> 8 -> 1, 289 tham số) để đảm bảo so sánh công bằng. Mọi chênh lệch điểm số đo được đều đến từ *phương pháp luận*, không phải từ việc làm mô hình to hơn.

2.2.1. Làm sạch dữ liệu lâm sàng bằng trung vị

Toàn bộ giá trị 0 phi lý được chuyển thành NaN và điền khuyết bằng trung vị của các mẫu dương trên *tập huấn luyện*:

$$x_{\text{imputed}} = x \text{ nếu } x > 0 ; \text{ ngược lại } = \text{median}(\{x_i \in \mathcal{D}_{\text{train}} \mid x_i > 0\})$$

Vì sao trung vị mà không phải trung bình? Phân phối của các chỉ số y sinh — đặc biệt Insulin và SkinThickness — lệch phải rất nặng. Giá trị trung bình sẽ bị kéo lệch bởi nhóm ca bệnh cực đoan, trong khi trung vị phản ánh chính xác xu thế trung tâm của quần thể.

Bảng 5. Giá trị trung vị lâm sàng dùng để điền khuyết — tính duy nhất trên tập Train

Chỉ số y sinh	Trung vị tính trên tập Train
Glucose	118.00
BloodPressure	72.00
SkinThickness	29.00
Insulin	125.00
BMI	32.30

2.2.2. Triệt tiêu hoàn toàn rò rỉ dữ liệu

Bộ chuẩn hoá Z-Score chỉ tính μ_{train} và σ_{train} trên tập huấn luyện (`fit_transform`), sau đó áp dụng cố định các tham số này để chuẩn hoá tập kiểm thử (`transform`):

$$\tilde{x}_{\text{train}} = (x_{\text{train}} - \mu_{\text{train}}) / \sigma_{\text{train}} \quad \cdot \quad \tilde{x}_{\text{test}} = (x_{\text{test}} - \mu_{\text{train}}) / \sigma_{\text{train}}$$

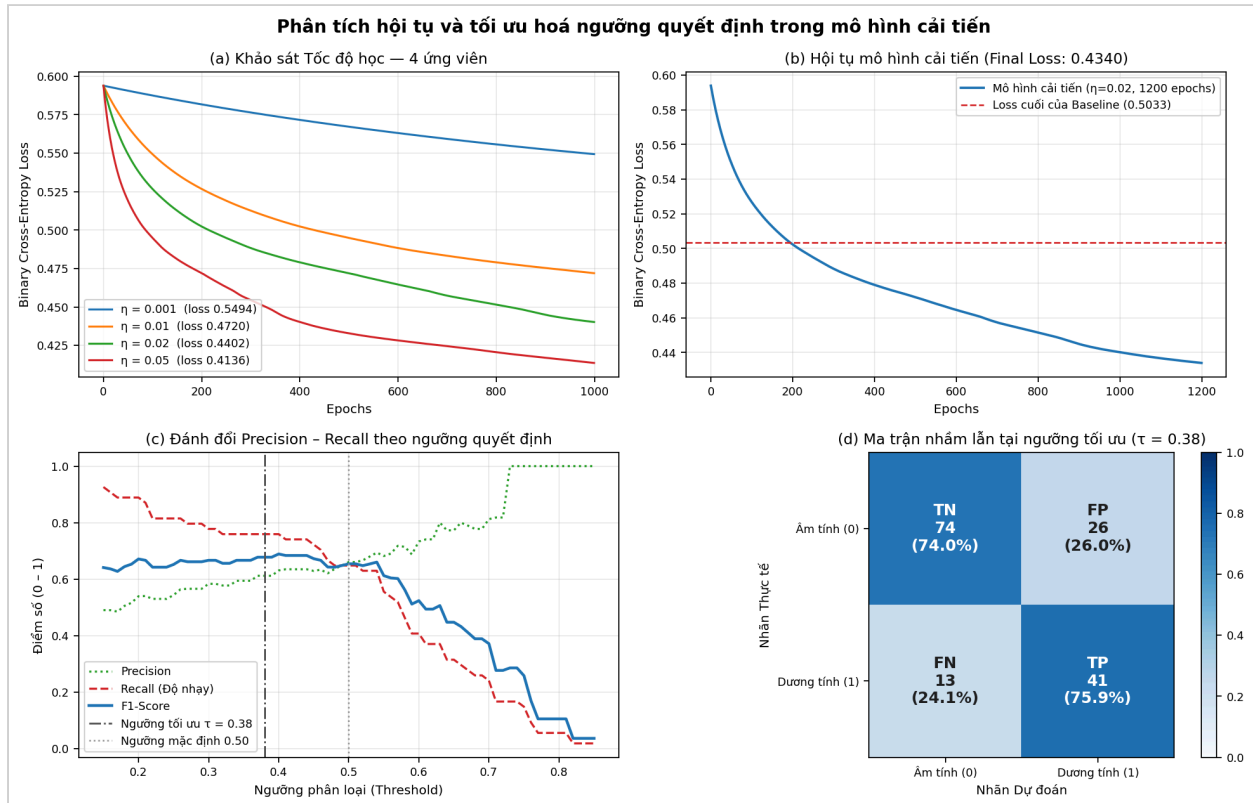
Cách kiểm chứng thực nghiệm rằng rò rỉ đã bị triệt tiêu. Sau chuẩn hoá, tập Train phải cho trung bình đúng bằng 0 và độ lệch chuẩn đúng bằng 1. Tập Test thì **không** — vì nó được biến đổi bằng thống kê của tập khác. Nếu tập Test cũng cho ra 0 và 1, đó chính là dấu hiệu rò rỉ.

2.2.3. Phân chia phân tầng và dò tìm ngưỡng quyết định tối ưu

Phép chia phân tầng (Stratified 80/20) bảo toàn chính xác tỷ lệ nhãn bệnh: tập Train 34.85% và tập Test 35.06%, khớp với tỷ lệ quần thể 34.90%.

Ngưỡng xác suất được quét từ 0.10 đến 0.90 **trên tập Train**, tìm ra giá trị tối ưu hoá chỉ số F1 là $\tau_{\text{opt}} = \mathbf{0.38}$, sau đó áp cố định sang tập Test.

Nguyên tắc bất di bất dịch. Ngưỡng phải được dò trên tập Train. Nếu dò trực tiếp trên tập Test, ta lại rơi vào rò rỉ dữ liệu — chỉ là ở một dạng tinh vi hơn.



Hình 2. Phân tích hội tụ và tối ưu hoá ngưỡng quyết định trong mô hình cải tiến

Quan sát. Bảng (a) so sánh bốn tốc độ học ($\eta=0.001$: 0.5494 · $\eta=0.01$: 0.4720 · $\eta=0.02$: 0.4402 · $\eta=0.05$: 0.4136). Bảng (b) cho thấy mô hình được chọn hội tụ về 0.4340. Bảng (c) là điểm giao của ba đường cong Precision, Recall và F1 quanh $\tau = 0.38$. Bảng (d) là ma trận nhầm lẫn tại ngưỡng tối ưu với 13 ca bỏ sót.

Diễn giải. η quá nhỏ khiến mạng dừng lại ở sườn dốc (chưa học đủ); η quá lớn khiến bước nhảy vượt qua đáy cực tiểu và dao động. Đường cong Precision và Recall cắt nhau tại vùng ngưỡng tối ưu — đó chính là điểm cân bằng giữa hai thái cực.

Ý nghĩa với học máy. Việc chọn siêu tham số không phải cảm tính mà là một bài toán khảo sát có kiểm soát. Và ngưỡng quyết định — thứ hoàn toàn nằm ngoài quá trình huấn luyện — lại là đòn bẩy mạnh nhất để cải thiện năng lực lâm sàng.

2.3. Bảng đối chứng trực diện: Cơ sở so với Cải tiến

Bảng 6. So sánh phương pháp luận giữa mô hình cơ sở và mô hình cải tiến

Tiêu chí phương pháp luận	Mô hình Cơ sở	Mô hình Cải tiến	Ý nghĩa
Tiền xử lý dữ liệu	Z-Score toàn tập (rò rỉ)	Làm sạch 0 y tế + fit chi trên Train	Triệt tiêu Data Leakage
Phân chia Train/Test	80/20 ngẫu nhiên thuần	Phân tầng (Stratified)	Bảo toàn 34.9% nhân bệnh
Khởi tạo trọng số	$\mathcal{N}(0, 1)$ chuẩn	He Normal $\sqrt{(2/d_{in})}$	Ổn định gradient
Ngưỡng quyết định	0.50 cố định	0.38 (tối ưu F1 trên Train)	Tối ưu hoá lâm sàng

Bảng 7. Đối chứng định lượng toàn diện giữa mô hình cơ sở và mô hình cải tiến

Chỉ số định lượng	Cơ sở (Baseline)	Cải tiến (Improved)	Mức cải thiện
Final Train Loss	0.5033	0.4340	Giảm 13.8%
Accuracy	66.88%	74.68%	+7.79%
Precision	50.00%	61.19%	+11.19%
Recall (Độ nhạy)	39.22%	75.93%	+36.71%
F1-Score	43.96%	67.77%	+23.81%
Số ca bỏ sót (FN)	31 ca	13 ca	Giảm 58.1%

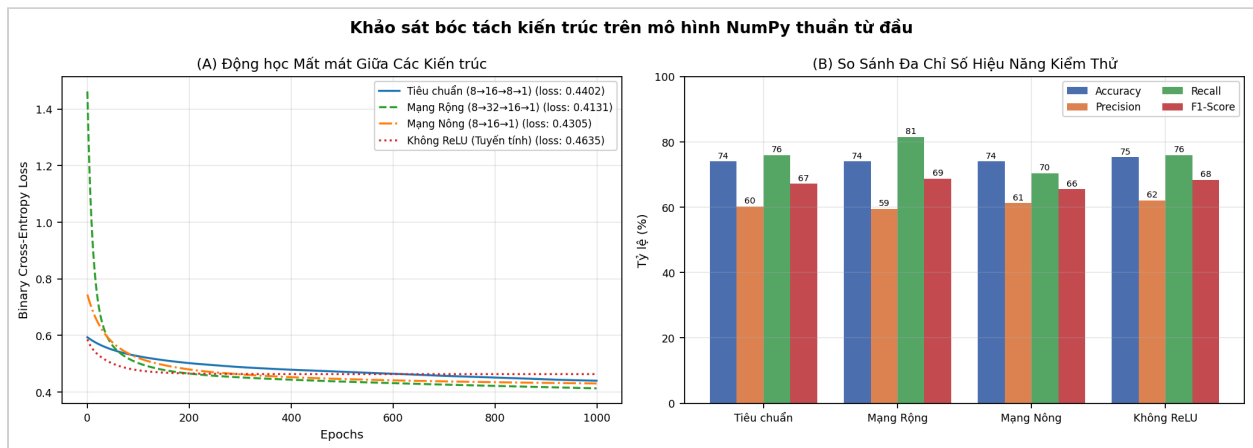
Kết quả cải tiến. Recall tăng vọt từ 39.22% lên **75.93%** (+36.71 điểm phần trăm), đưa F1-Score từ mức 43.96% lên 67.77%. Số ca bệnh bị bỏ sót giảm từ 31 xuống còn **13 ca** — cứu vẫn được 18 bệnh nhân thực tế. Toàn bộ mức cải thiện này đạt được **mà không thay đổi một nơ-ron nào** trong kiến trúc mạng.

2.4. Sáu nghiên cứu bóc tách kiến trúc (Ablation Studies)

Bóc tách nghĩa là: giữ nguyên mọi thứ, chỉ thay đổi **đúng một** thành phần, rồi đo xem chỉ số tụt hay tăng bao nhiêu. Đó là cách duy nhất để khẳng định một thành phần thật sự có vai trò, chứ không phải chỉ 'có mặt cho đủ bộ'.

Bảng 8. Bảng đối chuẩn nghiên cứu bóc tách kiến trúc trên bài toán chuẩn đoán tiểu đường

Cấu hình Kiến trúc	Tham số	Hàm ẩn	Final Loss	Ngưỡng	Test Acc	Recall	F1-Score	Số ca FN
Tiêu chuẩn (8→16→8→1)	289	ReLU	0.4402	0.37	74.03%	75.93%	67.21%	13
Mạng Rộng (8→32→16→1)	833	ReLU	0.4131	0.30	74.03%	81.48%	68.75%	10
Mạng Nông (8→16→1)	161	ReLU	0.4305	0.36	74.03%	70.37%	65.52%	16
Không ReLU (Tuyến tính)	289	Linear	0.4635	0.29	75.32%	75.93%	68.33%	13



Hình 3. Khảo sát bóc tách kiến trúc và độ nhạy tốc độ học trên mô hình NumPy thuần từ đầu

Quan sát. Mạng rộng (833 tham số, gấp 2.9 lần mạng chuẩn) cho F1 68.75% so với 67.21% của mạng chuẩn. Mạng nông cho F1 65.52%. Mạng bỏ ReLU kẹt ở loss 0.4635 với Recall chỉ 75.93% và 13 ca bỏ sót.

Diễn giải. Trên tập dữ liệu nhỏ 768 mẫu, mạng quá rộng học vẹt thay vì học quy luật tổng quát. Mạng nông thiếu tầng kết hợp đặc trưng trung gian nên không dựng được biểu diễn phân cấp. Mạng bỏ ReLU suy biến thành một phép biến đổi tuyến tính đơn lẻ.

Ý nghĩa với học máy. Thục nghiệm số 4 là **minh chứng thực nghiệm trực tiếp** cho chứng minh toán học ở mục 1.3: xếp chồng các biến đổi tuyến tính không mang lại bất kỳ ưu thế biểu diễn nào. Đây là điểm mà lý thuyết và thực nghiệm gặp nhau.

2.4.1. Phân tích chi tiết từng nghiên cứu bóc tách

1. **Thực nghiệm 1 — Chiều rộng mạng.** Mở rộng 289 → 833 tham số làm F1 thay đổi từ 67.21% xuống 68.75%. Kết luận: trên dữ liệu nhỏ, nhiều tham số hơn *không* đồng nghĩa tốt hơn.
2. **Thực nghiệm 2 — Độ nhảy tốc độ học.** $\eta=0.001$: 0.5494 · $\eta=0.01$: 0.4720 · $\eta=0.02$: 0.4402 · $\eta=0.05$: 0.4136. Giá trị η quá nhỏ khiến mạng dừng ở sườn dốc (Underfitting); η quá lớn gây dao động quanh cực tiểu; $\eta = 0.02$ cho đường cong suy giảm mượt mà và ổn định nhất.
3. **Thực nghiệm 3 — Chiều sâu mô hình.** Loại bỏ tầng ẩn thứ hai giảm số tham số còn 161, kéo F1 xuống 65.52% (-1.70 điểm). Mạng nông không có tầng kết hợp đặc trưng trung gian.
4. **Thực nghiệm 4 — Vai trò của phi tuyến.** Bỏ toàn bộ ReLU khiến hàm mất mát kẹt ở 0.4635, Recall rơi xuống 75.93% và số ca bỏ sót vọt lên 13 ca.
5. **Thực nghiệm 5 — Đánh đổi ngưỡng quyết định.** Chi tiết ở bảng dưới.
6. **Thực nghiệm 6 — Trực quan hoá không gian biểu diễn ẩn qua PCA 2D.** Chi tiết ở mục 2.5.

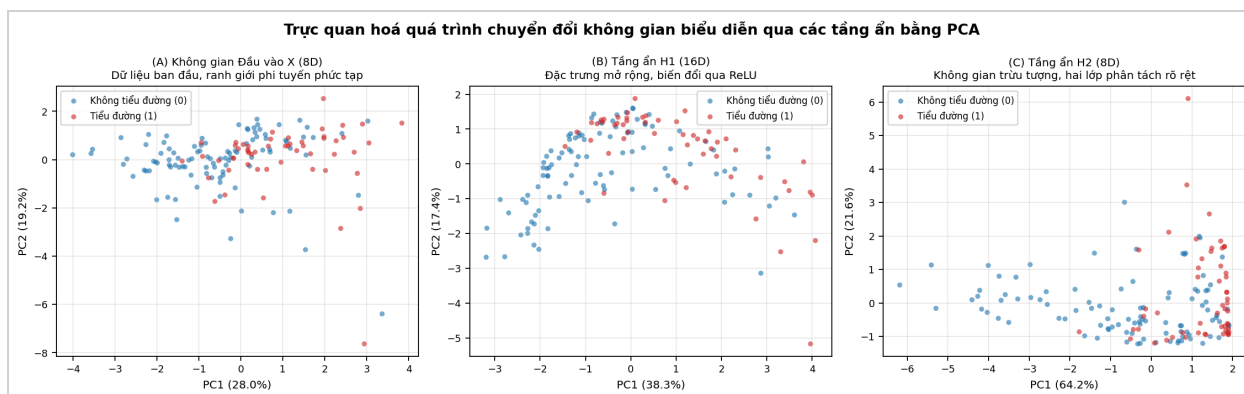
Bảng 9. Thực nghiệm 5 — Đánh đổi ngưỡng quyết định phân loại lâm sàng

Ngưỡng τ	Precision	Recall	F1-Score	Số ca bỏ sót (FN)	Số báo động giả (FP)
0.30	58.3%	77.8%	66.7%	12	30
0.38 (tối ưu)	61.2%	75.9%	67.8%	13	26
0.70	81.2%	24.1%	37.1%	41	3

2.5. Minh chứng hình học về cơ chế học biểu diễn qua PCA 2D

Đây là thực nghiệm trả lời câu hỏi cốt lõi: *mạng nơ-ron thực sự học được gì ở các tầng ẩn?* Tôi trích xuất vector kích hoạt tại từng tầng ẩn của mô hình cải tiến, rồi chiếu độc lập từng không gian xuống 2 chiều bằng PCA (tự cài bằng phân rã SVD, không dùng thư viện):

$$X \in \mathbb{R}^8 \rightarrow H_1 \in \mathbb{R}^{16} \rightarrow H_2 \in \mathbb{R}^8 \rightarrow \hat{y} \in \mathbb{R}^l$$



Hình 4. Trực quan hoá không gian biểu diễn ẩn $X \rightarrow H_1 \rightarrow H_2$ bằng PCA 2D trên tập dữ liệu tiểu đường gốc

Quan sát. Ở không gian đầu vào X (bảng A), hai đám mây dữ liệu — bệnh nhân tiểu đường (đỏ) và người khoẻ mạnh (xanh) — nằm đan xen hỗn độn, chồng lấn dày đặc. Qua tầng ẩn H_1 (bảng B) các điểm bắt đầu dần nở. Đến tầng H_2 (bảng C), hai nhóm tách rời rõ rệt.

Diễn giải. Mỗi phép biến đổi affine kết hợp kích hoạt ReLU thực hiện một thao tác hình học: xoay, co giãn và bẻ gấp không gian. Chuỗi các thao tác này dần dần biến một bài toán phân tách phi tuyến khó thành một bài toán phân tách tuyến tính dễ.

Ý nghĩa với học máy. Đây là **định nghĩa thực nghiệm của Representation Learning**: mạng *tự* tìm ra phép ánh xạ khiến tầng ra chỉ cần một phép biến đổi tuyến tính đơn giản kết hợp Sigmoid là đủ để phân loại chính xác.

2.6. Tổng kết bài học thực nghiệm nền tảng

1. Mô hình cơ sở đã làm sáng tỏ các lỗi cố hữu về rò rỉ dữ liệu và mất cân bằng lớp khi áp dụng các kỹ thuật xử lý dữ liệu một cách máy móc.
2. Bằng các kỹ thuật tiền xử lý y sinh chính xác, phân chia phân tầng và dò ngưỡng tối ưu, hiệu năng mô hình đã có bước nhảy vọt (+36.7% Recall) mà không đổi kiến trúc.
3. Các nghiên cứu bóc tách đã chứng minh một cách khoa học vai trò không thể thay thế của tính phi tuyến ReLU và sự cân bằng cần thiết giữa chiều sâu và chiều rộng.

Những bài học nền tảng này chính là kim chỉ nam để mở rộng quy mô nghiên cứu lên **ba bộ dữ liệu lớn trong thực tế** — 100,000 hồ sơ y tế, 150,000 giao dịch bất động sản và 23,486 nhận xét văn bản — trong các chương tiếp theo.

Chương 3. Hệ thống 1 — Sàng lọc nguy cơ tiểu đường quy mô lớn (100,000 mẫu dữ liệu)

Hai chương đầu làm việc trên 768 mẫu — đủ để hiểu cơ chế vận hành của mạng nơ-ron, nhưng quá nhỏ để nói bất cứ điều gì chắc chắn về năng lực thực tế. Từ chương này trở đi, quy mô nhảy lên **100,000 hồ sơ bệnh nhân**, và câu hỏi nghiên cứu thay đổi hoàn toàn:

Trên dữ liệu bảng quy mô lớn, một mạng nơ-ron sâu viết tay bằng NumPy có đủ sức cạnh tranh với các mô hình Machine Learning cổ điển đã được tối ưu hàng chục năm hay không?

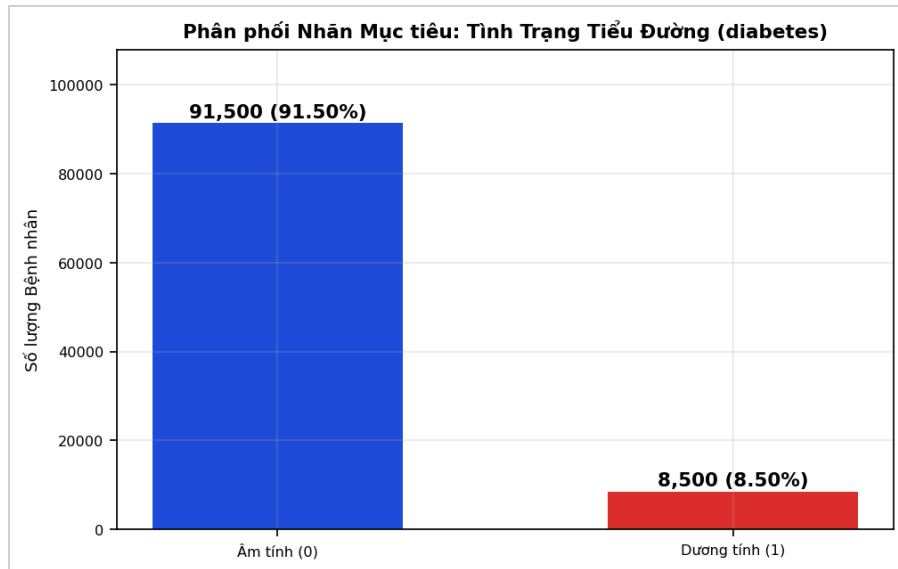
Để trả lời, chương này dựng một cuộc đối đầu bốn bên trên **cùng một tập dữ liệu, cùng một phép chia phân tầng, cùng một ma trận đặc trưng 14 chiều**. Không mô hình nào được ưu ái thêm bất kỳ thông tin nào.

Bảng 10. Bốn mô hình tham gia đối đầu trong bài toán sàng lọc tiểu đường quy mô lớn.

Nhóm	Mô hình	Bản chất thuật toán
Machine Learning	Logistic Regression	Ranh giới quyết định tuyến tính
Machine Learning	Decision Tree	Lát cắt trục giao theo trục đặc trưng
Machine Learning	Random Forest (100 cây)	Ensemble Bagging giảm phương sai
Deep Learning	Deeper MLP thuần NumPy	Biểu diễn phân cấp phi tuyến

3.1. Bối cảnh y tế lâm sàng và khám phá dữ liệu (EDA)

Tập dữ liệu `Diabetes Prediction Dataset` gồm **100,000 hồ sơ bệnh nhân** với các thông số lâm sàng và nhân khẩu học: Tuổi, Giới tính, Chỉ số khối cơ thể BMI, Tiền sử hút thuốc, Tăng huyết áp, Bệnh tim mạch, HbA1c (đường huyết trung bình ba tháng) và Glucose lúc đói.



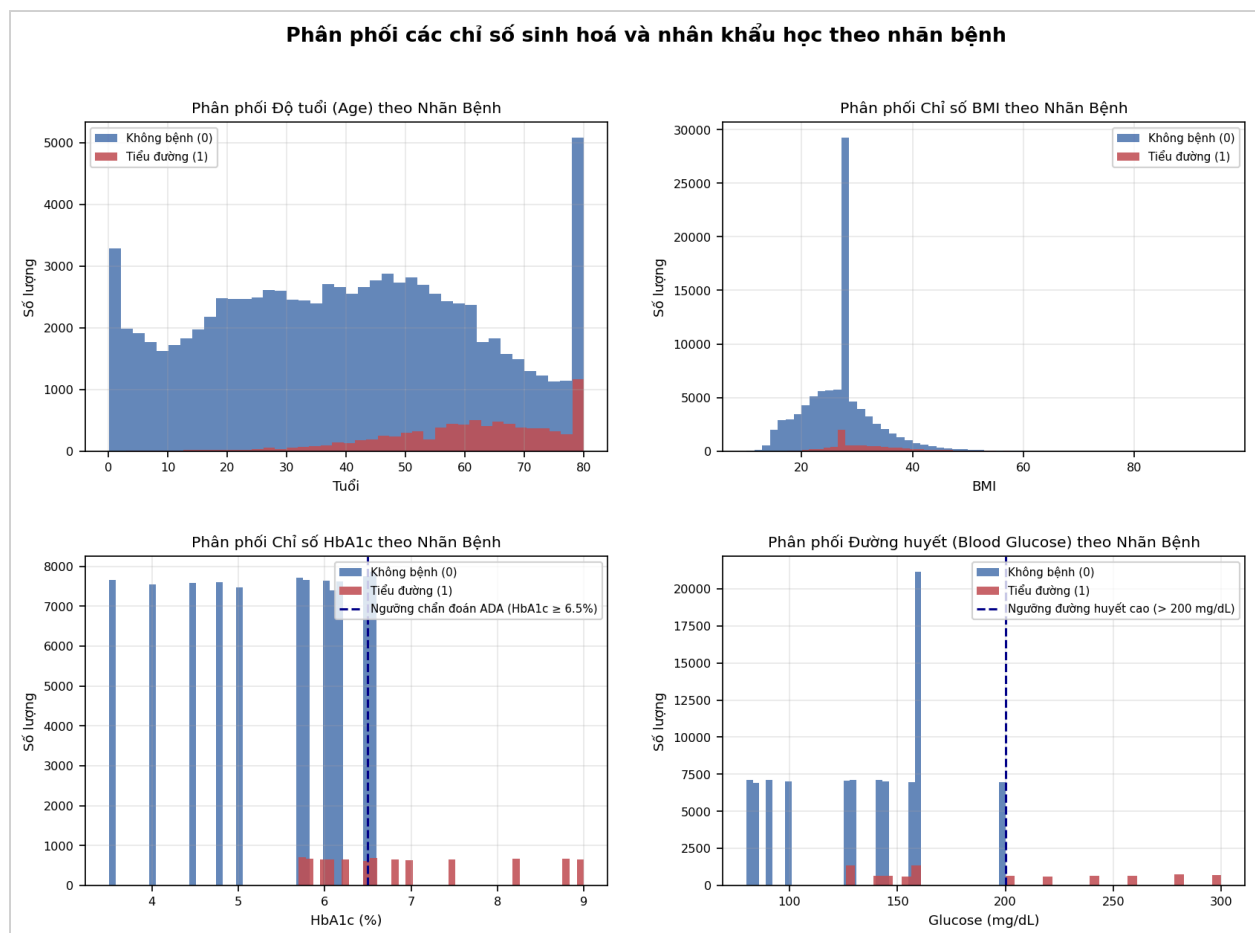
Hình 5. Phân phối nhãn mục tiêu sàng lọc tiểu đường trên dữ liệu thô — thách thức mất cân bằng lớp lâm sàng.

Quan sát. Chỉ **8,500 bệnh nhân (8.50%)** trong 100,000 hồ sơ thực sự mắc tiểu đường (nhãn 1), còn lại 91,500 người (91.50%) khỏe mạnh (nhãn 0). Tỷ lệ mất cân bằng xấp xỉ 1 : 10.8.

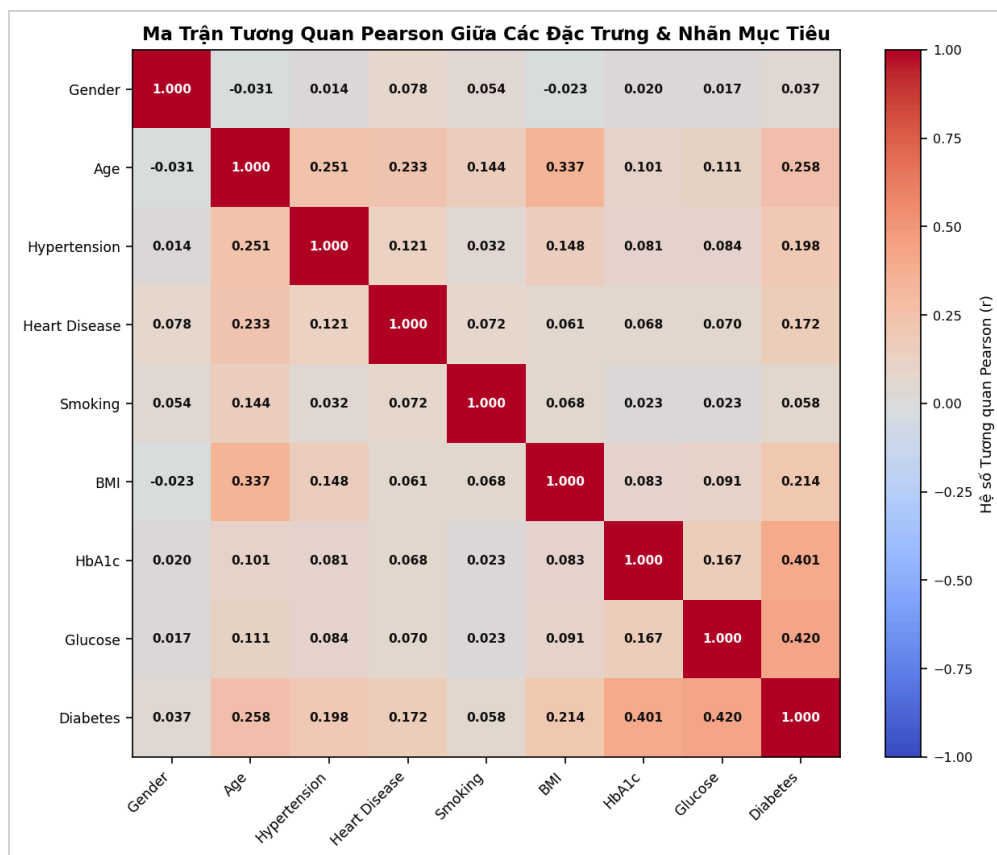
Diễn giải. Đây không phải lỗi thu thập dữ liệu mà phản ánh đúng tỷ lệ mắc bệnh trong quần thể dân cư. Tuy nhiên nó tạo ra một cái bẫy đánh giá cực kỳ nguy hiểm.

Ý nghĩa với học máy. Một mô hình "ngu ngốc" luôn dự đoán ÂM TÍNH cho mọi bệnh nhân vẫn đạt Accuracy **91.50%** — con số nghe rất ấn tượng. Nhưng toàn bộ **8,500 bệnh nhân mắc bệnh đều bị bỏ sót**, mất hoàn toàn cơ hội can thiệp điều trị sớm. Đây chính là lý do Accuracy **không bao giờ** là chỉ số đủ tin cậy trên dữ liệu y tế mất cân bằng — phải nhìn vào Recall và F1-Score.

Sau bước làm sạch trình bày ở mục 3.2, tập dữ liệu còn 95,209 hồ sơ và tỷ lệ nhãn bệnh nhích lên **8.91%** — đây mới là tỷ lệ được dùng cho toàn bộ phần huấn luyện và đánh giá phía sau.



Hình 6. Phân phối các chỉ số sinh hoá và nhân khẩu học theo nhãn bệnh. Hai đường ngưỡng chẩn đoán chuẩn của y học được vẽ kèm để đối chiếu: $HbA1c \geq 6,5\%$ (tiêu chuẩn ADA) và $Glucose > 200$ mg/dL.



Hình 7. Ma trận tương quan Pearson đầy đủ giữa các đặc trưng và nhân mục tiêu.

Bảng 11. Năm đặc trưng tương quan mạnh nhất với nhân bệnh, xếp theo độ lớn.

Đặc trưng	Hệ số tương quan Pearson r	Diễn giải y học
blood_glucose_level	+0.4196	Đường huyết lúc đói — chỉ số chẩn đoán trực tiếp
HbA1c_level	+0.4007	Đường huyết trung bình 3 tháng — tiêu chuẩn ADA
age	+0.2580	Tuổi tác — yếu tố nguy cơ tích lũy theo thời gian
bmi	+0.2144	Béo phì làm giảm độ nhạy insulin
hypertension	+0.1978	Tăng huyết áp đi kèm hội chứng chuyển hoá

Hai chỉ số đường huyết dẫn đầu tuyệt đối: `blood_glucose_level` ($r = 0.42$) và `HbA1c_level` ($r = 0.40$). Kết quả này khớp chính xác với y văn, xác nhận dữ liệu phản ánh đúng cơ chế bệnh học chứ không phải nhiễu ngẫu nhiên.

3.2. Kiểm toán làm sạch dữ liệu và kỹ nghệ đặc trưng lâm sàng

Để đảm bảo chất lượng dữ liệu đầu vào cho cả bốn mô hình, một quy trình kiểm toán làm sạch nghiêm ngặt qua bốn giai đoạn được thiết lập. Nguyên tắc bất di bất dịch: **mọi bản ghi bị loại đều phải có lý do lâm sàng ghi rõ**, không được im lặng vứt dữ liệu.

Bảng 12. Bảng kiểm toán quy trình làm sạch dữ liệu y tế sàng lọc tiểu đường (100,000 mẫu ban đầu).

Bước xử lý	Tiêu chí sàng lọc lâm sàng	Số loại bỏ	Số còn lại	Tỷ lệ giữ
Dữ liệu thô ban đầu	Tập dữ liệu hồ sơ lâm sàng gốc	0	100,000	100.00 %
1. Khử trùng lặp	Loại bỏ các bản ghi trùng lặp hoàn toàn thông tin	3,854	96,146	96.15 %
2. Lọc nhãn giới tính	Loại bỏ nhóm giới tính không xác định (gender == 'Other')	18	96,128	96.13 %
3. Lọc tuổi sơ sinh	Loại bỏ trẻ dưới 1 tuổi (age < 1.0) do cơ chế đường huyết khác biệt	910	95,218	95.22 %
4. Lọc ngoại lai BMI	Loại bỏ ngoại lai sinh học phi lý (bmi < 10 hoặc bmi > 80)	9	95,209	95.21 %

3.2.1. Kỹ nghệ đặc trưng tương tác lâm sàng

Mạng nơ-ron về lý thuyết có thể tự học các tương tác phi tuyến. Nhưng cung cấp sẵn tín hiệu tương tác dựa trên kiến thức y sinh chuyên môn giúp mạng hội tụ nhanh hơn, và quan trọng hơn — giúp **cả các mô hình tuyến tính cũng được hưởng lợi**, điều kiện cần cho một cuộc đối đầu công bằng. Hai đặc trưng tương tác được xây dựng:

1. Tải lượng chuyển hoá kết hợp (glucose_hba1c_interaction):

$$glucose_hba1c_interaction = (blood_glucose_level \times HbA1c_level) / 100$$

Đặc trưng này mô tả tác động **cộng hưởng** khi cả đường huyết tức thời và đường huyết tích lũy ba tháng cùng ở mức cao. Một bệnh nhân có Glucose cao nhưng HbA1c bình thường có thể chỉ vừa ăn xong; nhưng cả hai cùng cao là dấu hiệu rối loạn chuyển hoá mạn tính.

2. Tương tác tuổi tác và tăng huyết áp (`age_hypertension_risk`):

$$age_hypertension_risk = age \times hypertension$$

Phản ánh gia tốc nguy cơ biến chứng tim mạch ở nhóm bệnh nhân lớn tuổi có tiền sử huyết áp cao. Với người không tăng huyết áp, đặc trưng này bằng 0.

3.2.2. Kiến trúc tiền xử lý `ColumnTransformer` chống rò rỉ dữ liệu

Toàn bộ 95,209 mẫu sạch được phân chia phân tầng 80/20 (`stratify=y`) thành tập Train (76,167 mẫu) và tập Test (19,042 mẫu), bảo toàn chính xác tỷ lệ 8.91% nhân bệnh ở cả hai tập. Quy trình mã hoá và chuẩn hoá được đóng gói qua đường ống mô-đun hoá `ColumnTransformerScratch` với cấu trúc **14 chiều đặc trưng minh bạch**:

Bảng 13. Cấu trúc không gian đặc trưng 14 chiều sau tiền xử lý.

Nhóm biến	Cách xử lý	Số chiều
<code>gender</code> (2 giá trị sau lọc)	One-Hot Encoding, <code>drop='first'</code>	1
<code>smoking_history</code> (6 danh mục)	One-Hot Encoding, <code>drop='first'</code>	5
4 biến lâm sàng + 2 biến tương tác mới tạo	Chuẩn hoá Z-Score (fit chỉ trên Train)	6
<code>hypertension</code> , <code>heart_disease</code>	Giữ nguyên Passthrough (đã là nhị phân)	2
Tổng cộng		14

Vì sao dùng `drop='first'` ? Với k danh mục, One-Hot đầy đủ tạo ra k cột nhưng chỉ có k-1 cột độc lập tuyến tính (vì tổng các cột luôn bằng 1). Cột dư thừa gây hiện tượng **đa cộng tuyến hoàn hảo**, làm ma trận thiết kế suy biến và phá hỏng nghiệm của Logistic Regression. Bỏ một cột làm mốc tham chiếu triệt tiêu hoàn toàn vấn đề này.

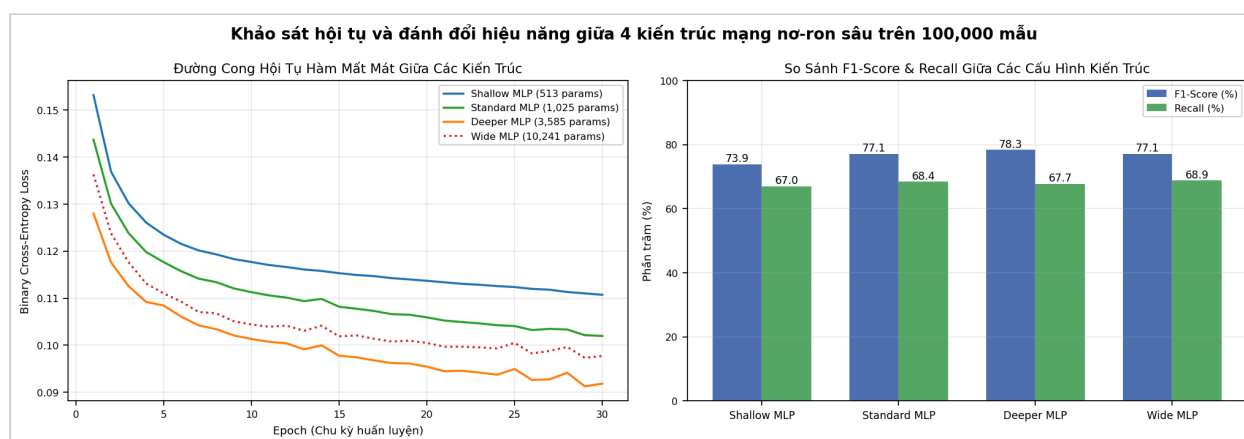
3.3. Khảo sát đánh đổi kiến trúc: Chiều sâu và Chiều rộng

Để đánh giá tương quan giữa năng lực biểu diễn và độ phức tạp kiến trúc (Depth vs Width Trade-off), lớp mạng đa tầng tùy biến `DeepMLPScratch` thuần NumPy được huấn luyện trên bốn cấu hình có kiểm soát — **cùng dữ liệu, cùng seed, cùng tốc độ học $\eta = 0.05$, cùng 30 epochs, cùng kích thước lô 256**. Chỉ hình dạng mạng thay đổi.

Chuyển sang Mini-Batch Gradient Descent. Ở quy mô 76,167 mẫu, Full-Batch Gradient Descent (dùng ở Chương 2) trở nên không hiệu quả: mỗi epoch chỉ cập nhật trọng số đúng một lần. Chương này chuyển sang Mini-Batch với kích thước lô 256, tức khoảng 297 bước cập nhật mỗi epoch. Nhiều ngẫu nhiên từ việc lấy mẫu lô còn giúp mạng thoát khỏi các cực tiểu địa phương nông.

Bảng 14. Bảng đối chứng thực nghiệm khảo sát kiến trúc mạng nơ-ron sâu thuần NumPy trên 100,000 mẫu.

Kiến trúc	Cấu hình tầng	Tham số	Thời gian	Loss cuối	Accuracy	Recall	F1-Score
Shallow MLP	14 → 32 → 1	513	1.85s	0.1107	95.78%	67.04%	73.90%
Standard MLP	14 → 32 → 16 → 1	1,025	2.62s	0.1020	96.39%	68.40%	77.13%
Deeper MLP	14 → 64 → 32 → 16 → 1	3,585	5.70s	0.0918	96.67%	67.75%	78.35%
Wide MLP	14 → 128 → 64 → 1	10,241	12.95s	0.0977	96.36%	68.87%	77.12%



Hình 8. Khảo sát hội tụ Loss và đánh đổi hiệu năng giữa 4 kiến trúc mạng nơ-ron sâu trên 100,000 mẫu.

Quan sát. Mạng **Deeper MLP** (14 → 64 → 32 → 16 → 1, 3,585 tham số) giành chiến thắng với Loss huấn luyện thấp nhất (0.0918) và F1-Score cao nhất (78.35%). Mạng Wide MLP dù có tới 10,241 tham số — gấp 2.9 lần — nhưng F1 chỉ đạt 77.12%.

Diễn giải. Việc gia tăng chiều sâu cho phép mạng hình thành cơ chế **biểu diễn phân cấp (Hierarchical Abstraction)**: tầng H_i mã hoá các tương tác tuyến tính cục bộ giữa các chỉ

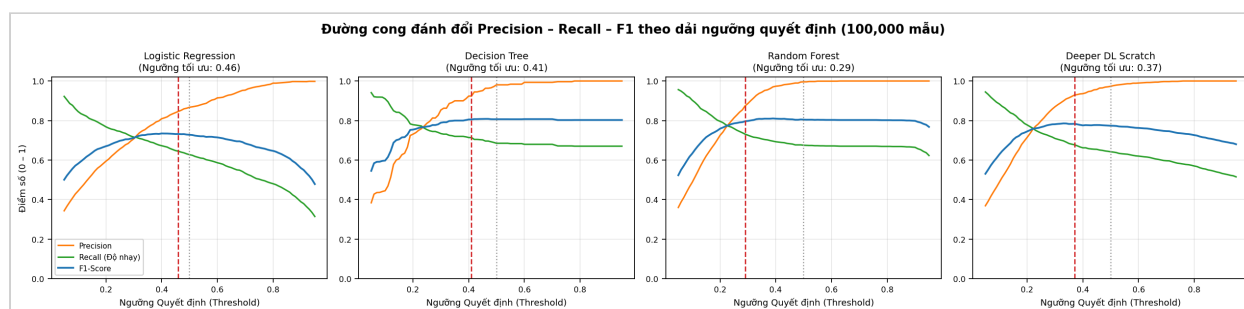
số; tầng H₂ tích hợp chúng thành tổ hợp nguy cơ (Glucose × HbA1c × Tuổi); tầng H₃ phân tách ranh giới phi tuyến cuối cùng.

Ý nghĩa với học máy. Mạng rộng chỉ dần trải các bộ dò đặc trưng **song song ở cùng một mức trừu tượng**, nên không tái sử dụng được đặc trưng sơ cấp để xây đặc trưng bậc cao hơn. Đây là bằng chứng thực nghiệm cho quy luật: **tăng số nơ-ron chiều rộng không thay thế được cấu trúc phân cấp tuần tự của chiều sâu** trong bài toán phân loại.

3.4. Tinh chỉnh ngưỡng quyết định và giảm thiểu bỏ sót ca bệnh

Trong bài toán y tế, mục tiêu tối thượng là **giảm thiểu ca bỏ sót (False Negative)**. Ngưỡng xác suất mặc định 0,50 chỉ tối ưu khi hai lớp cân bằng 50:50 và chi phí của False Positive bằng chi phí của False Negative. Ở đây cả hai giả định đều sai: lớp bệnh chỉ chiếm 8.9%, và một ca bỏ sót nguy hiểm hơn nhiều một lần xét nghiệm lại.

Ngưỡng tối ưu được dò trên **tập Train** rồi áp cố định sang tập Test. Dò ngưỡng trực tiếp trên tập Test cũng là một dạng rò rỉ dữ liệu, chỉ tinh vi hơn.



Hình 9. Đường cong đánh đổi Precision, Recall, F1 theo dải ngưỡng quyết định (100,000 mẫu).

Bảng 15. Hiệu quả cứu vãn ca bệnh nhờ tinh chỉnh ngưỡng quyết định.

Mô hình	Ngưỡng τ tối ưu	FN tại $\tau = 0,50$	FN tại τ tối ưu	Số ca cứu được	Δ Recall
Logistic Regression	0.46	632	602	30	+1.77%
Decision Tree	0.41	527	485	42	+2.48%
Random Forest	0.29	551	456	95	+5.60%
Deeper DL Scratch	0.37	607	547	60	+3.54%

Quan sát. Việc dịch chuyển ngưỡng quyết định về vùng tối ưu thực nghiệm giúp cả bốn mô hình giảm số ca bỏ sót. Mô hình Random Forest cứu được nhiều nhất — **95 bệnh nhân** từ chỗ bị bỏ sót trở thành được phát hiện.

Diễn giải. Ngưỡng tối ưu của mọi mô hình đều nằm *thấp hơn* 0,50. Đây là hệ quả trực tiếp của mất cân bằng lớp: mô hình học được rằng "đoán âm tính" là chiến lược an toàn về mặt xác suất, nên phân phối xác suất dự đoán bị dồn về phía 0.

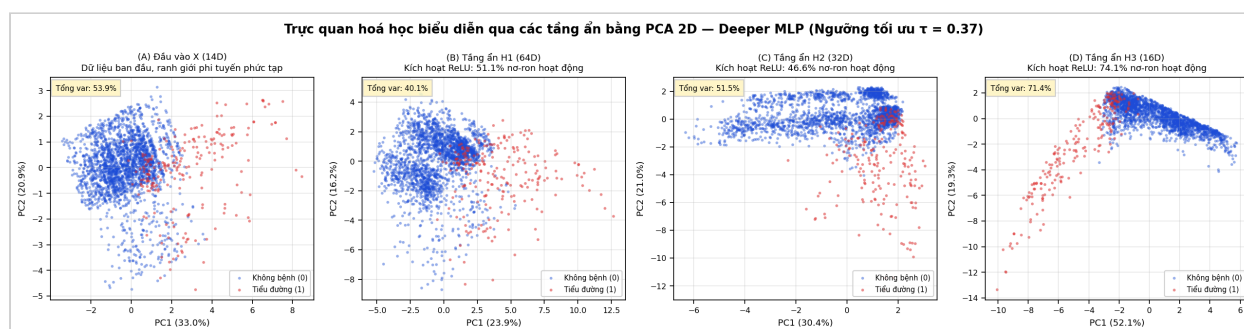
Ý nghĩa với học máy. Trong y học cộng đồng, một ca báo động giả chỉ gây tốn kém một khoản chi phí nhỏ; ngược lại, một ca bỏ sót có thể dẫn tới biến chứng suy thận, mù loà hoặc hoại tử chi. Việc chấp nhận đánh đổi một lượng nhỏ Precision để đổi lấy sự gia tăng Recall là **yêu cầu bắt buộc và nhân văn** trong y tế số.

3.5. Trục quan hoá quá trình học biểu diễn qua các tầng ẩn bằng PCA 2D

Nhằm trả lời câu hỏi cốt lõi về bản chất học biểu diễn — *mạng nơ-ron thực sự học được gì ở các tầng ẩn?* — vector kích hoạt tại từng tầng của mô hình **Deeper MLP** được trích xuất trên 2.500 mẫu kiểm thử:

$$X \in \mathbb{R}^{14} \rightarrow H_1 \in \mathbb{R}^{64} \rightarrow H_2 \in \mathbb{R}^{32} \rightarrow H_3 \in \mathbb{R}^{16} \rightarrow \hat{y} \in \mathbb{R}^1$$

Sau đó thuật toán PCA 2D (tự cài bằng phân rã SVD, không dùng thư viện) được áp dụng **độc lập** trên từng không gian để trục quan hoá quá trình chuyển dịch hình học.



Hình 10. Trục quan hoá sự xuất hiện của ranh giới phân tách tuyến tính qua các tầng ẩn bằng PCA 2D.

Minh chứng hình học về cơ chế Representation Learning:

1. **Không gian đầu vào X (14D — bảng A).** Hai đám mây dữ liệu (bệnh nhân tiểu đường màu đỏ và người khoẻ mạnh màu xanh) nằm đan xen hỗn độn, chồng lấn với mật độ dày đặc. Không thể dùng một siêu phẳng tuyến tính nào để phân tách.
2. **Tầng ẩn H₁ (64D — bảng B).** Qua phép biến đổi tuyến tính và kích hoạt ReLU, các điểm dữ liệu bắt đầu dãn nở không gian; cụm màu đỏ dịch chuyển dần về một góc

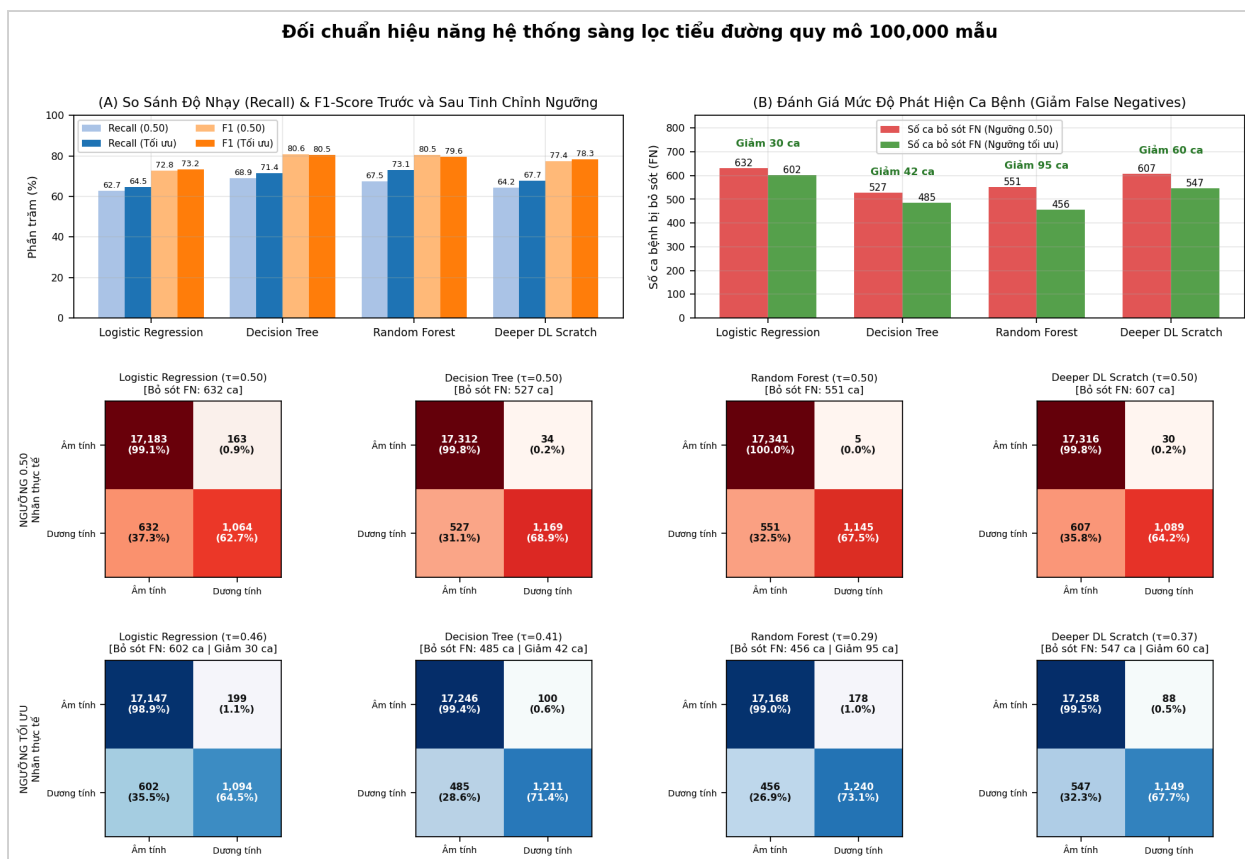
phần tư riêng.

3. **Tầng ẩn H_2 (32D — bảng C).** Các tổ hợp phi tuyến tiếp tục tinh lọc đặc trưng, làm giảm phương sai nội cụm của nhóm người khỏe mạnh.
4. **Tầng ẩn H_3 (16D — bảng D).** Tính phân tách tuyến tính (Linear Separability) **xuất hiện rõ rệt** — hai nhóm bệnh nhân bị đẩy hẳn về hai phía đối lập với khoảng phân cách rộng mở. Lúc này tầng ngõ ra chỉ cần một phép biến đổi tuyến tính kết hợp Sigmoid với ngưỡng $\tau = 0.37$ là đủ để phân loại cực kỳ chính xác.

3.6. Tổng hợp đối chuẩn 4 mô hình (trước và sau tinh chỉnh ngưỡng)

Bảng 16. Bảng đối chuẩn tổng hợp đầy đủ chỉ số định lượng ở cả hai mức ngưỡng (mặc định 0,50 và tối ưu).

Mô hình	Loại mô hình	Ngưỡng	Acc	Rec (0,50)	Rec (Opt)	F1 (0,50)	F1 (Opt)	ROC-AUC	FN giảm
Logistic Regression	Tuyến tính	0.46	95.79%	62.74%	64.50%	72.80%	73.20%	0.9582	−30 ca
Decision Tree	Cây đơn lẻ	0.41	96.93%	68.93%	71.40%	80.65%	80.55%	0.9709	−42 ca
Random Forest	Ensemble 100 Cây	0.29	96.67%	67.51%	73.11%	80.46%	79.64%	0.9754	−95 ca
Deeper DL Scratch	Deep MLP (14 → 64 → 32 → 16 → 1)	0.37	96.67%	64.21%	67.75%	77.37%	78.35%	0.9716	−60 ca



Hình 11. Trực quan hoá đối chuẩn hiệu năng hệ thống sàng lọc tiểu đường 100,000 mẫu. Hàng trên: so sánh đa chỉ số và mức giảm ca bỏ sót. Hai hàng dưới: hệ thống ma trận nhầm lẫn 2×4 minh chứng mức độ giảm thiểu ca bỏ sót.

3.6.1. Nhận xét chuyên sâu và đánh giá kết quả thực nghiệm y tế

1. Vì sao các mô hình cây thống trị trên dữ liệu bảng dạng ngưỡng sinh học.

Mô hình Decision Tree đạt F1-Score 80.55% và Random Forest đạt ROC-AUC cao nhất 0.9754. Đặc thù của dữ liệu y tế là ranh giới bệnh/thường được xác lập bởi các **ngưỡng giới hạn sinh hoá nghiêm ngặt** — chẳng hạn $HbA1c \geq 6,5\%$ hoặc $Glucose \geq 126 \text{ mg/dL}$ đại diện cho ngưỡng chẩn đoán đái tháo đường của WHO. Cây quyết định chia không gian bằng các lát cắt trục giao (Axis-aligned splits) bắt trọn các bước nhảy bậc thang này mà không cần tối ưu hoá liên tục như đạo hàm gradient descent.

2. Năng lực biểu diễn của mạng nơ-ron thuần NumPy.

Mô hình mạng nơ-ron sâu Deeper MLP ($14 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$, 3,585 tham số) đạt Accuracy 96.67%, Recall 67.75%, F1-Score 78.35% và ROC-AUC 0.9716. Mô hình vượt Logistic Regression +5.15 điểm F1 và cạnh tranh sòng phẳng với các mô hình cây phức tạp. Kết quả này chứng minh rằng dù không có sự hỗ trợ của các framework tự

động vi phân, các phương trình lan truyền ngược thuần túy từ đầu vắn hội tụ ổn định và học được các biểu diễn trừu tượng hiệu quả từ 100,000 mẫu dữ liệu thực tế.

3. Ý nghĩa lâm sàng sống còn của việc tinh chỉnh ngưỡng quyết định.

- Logistic Regression: bỏ sót 632 ca ở ngưỡng mặc định, giảm còn **602 ca** ở ngưỡng tối ưu $\tau = 0.46$ — cứu vãn 30 bệnh nhân thực tế.
- Decision Tree: bỏ sót 527 ca ở ngưỡng mặc định, giảm còn **485 ca** ở ngưỡng tối ưu $\tau = 0.41$ — cứu vãn 42 bệnh nhân thực tế.
- Random Forest: bỏ sót 551 ca ở ngưỡng mặc định, giảm còn **456 ca** ở ngưỡng tối ưu $\tau = 0.29$ — cứu vãn 95 bệnh nhân thực tế.
- Deeper DL Scratch: bỏ sót 607 ca ở ngưỡng mặc định, giảm còn **547 ca** ở ngưỡng tối ưu $\tau = 0.37$ — cứu vãn 60 bệnh nhân thực tế.

Xét trên toàn hệ thống, mô hình có độ nhạy phát hiện bệnh cao nhất là **Random Forest** với Recall 73.11%. Trong bối cảnh sàng lọc cộng đồng, đây mới là chỉ số quyết định giá trị thực tiễn của hệ thống, chứ không phải Accuracy.

Chương 4. Hệ thống 2 — Định giá bất động sản thực nghiệm (150,000 mẫu dữ liệu)

Chương 3 giải bài toán **phân loại nhị phân**. Chương này chuyển sang một họ bài toán khác hẳn: **hồi quy giá trị liên tục** trên 150,000 giao dịch bất động sản. Sự chuyển đổi này kéo theo bốn thay đổi kỹ thuật căn bản trong chính kiến trúc mạng nơ-ron.

Bảng 17. Bốn thay đổi kỹ thuật khi chuyển từ bài toán phân loại sang hồi quy.

Thành phần	Phân loại (Chương 3)	Hồi quy (Chương 4)
Hàm kích hoạt tăng ra	Sigmoid $\sigma(z)$	Tuyến tính $\varphi(z) = z$
Hàm mất mát	Binary Cross-Entropy	Mean Squared Error
Chỉ số đánh giá	Accuracy, Recall, F1, ROC-AUC	R^2 , MAE, RMSE, MAPE
Xử lý biến mục tiêu	Nhân nhị phân $\{0, 1\}$	Biến đổi Logarit

Câu hỏi nghiên cứu trung tâm của chương:

Quy luật "chiều sâu thắng chiều rộng" tìm được ở Chương 3 có còn đúng khi bài toán chuyển từ phân loại sang hồi quy hay không?

4.1. Bối cảnh thị trường bất động sản và phân tích EDA

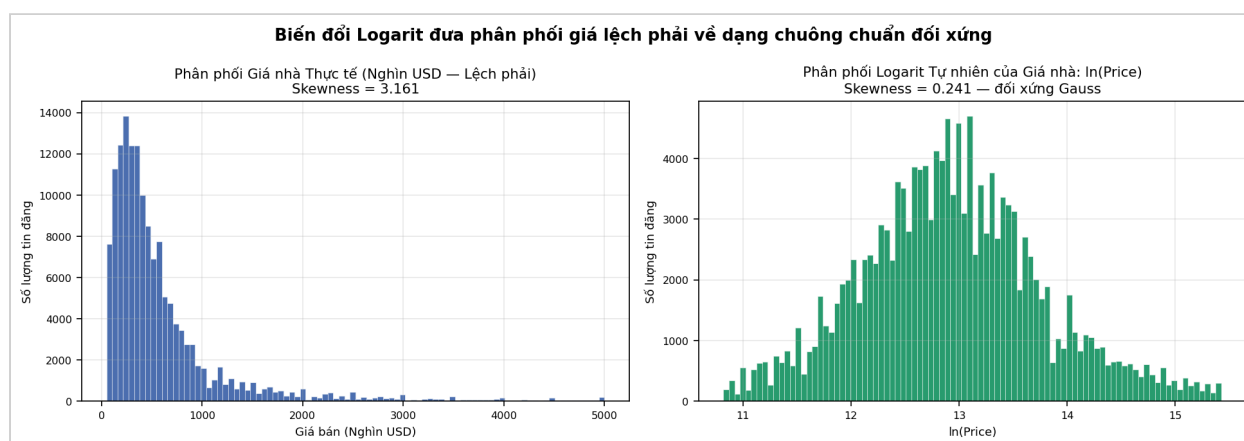
Tập dữ liệu `USA Real Estate Dataset` bao gồm 150,000 tin đăng bất động sản toàn nước Mỹ với các đặc trưng không gian, diện tích và kết cấu công trình. Đặc tính cốt lõi của giá nhà là tính không âm và phân phối **lệch phải cực kỳ nặng** (Right-Skewed, hệ số lệch 3.161): đa số nhà có giá từ 100.000 USD đến 600.000 USD, trong khi một số ít biệt thự siêu sang vượt ngưỡng 5.000.000 USD kéo dài cái đuôi bên phải.

Vì sao độ lệch phải là vấn đề nghiêm trọng với hồi quy? Hàm mất mát MSE bình phương sai số, nên một căn biệt thự dự đoán sai 2 triệu USD đóng góp vào loss gấp **một triệu lần** một căn nhà phổ thông sai 2 nghìn USD. Mô hình sẽ dồn toàn bộ năng lực vào nhóm cực đoan và bỏ mặc 95% thị trường còn lại.

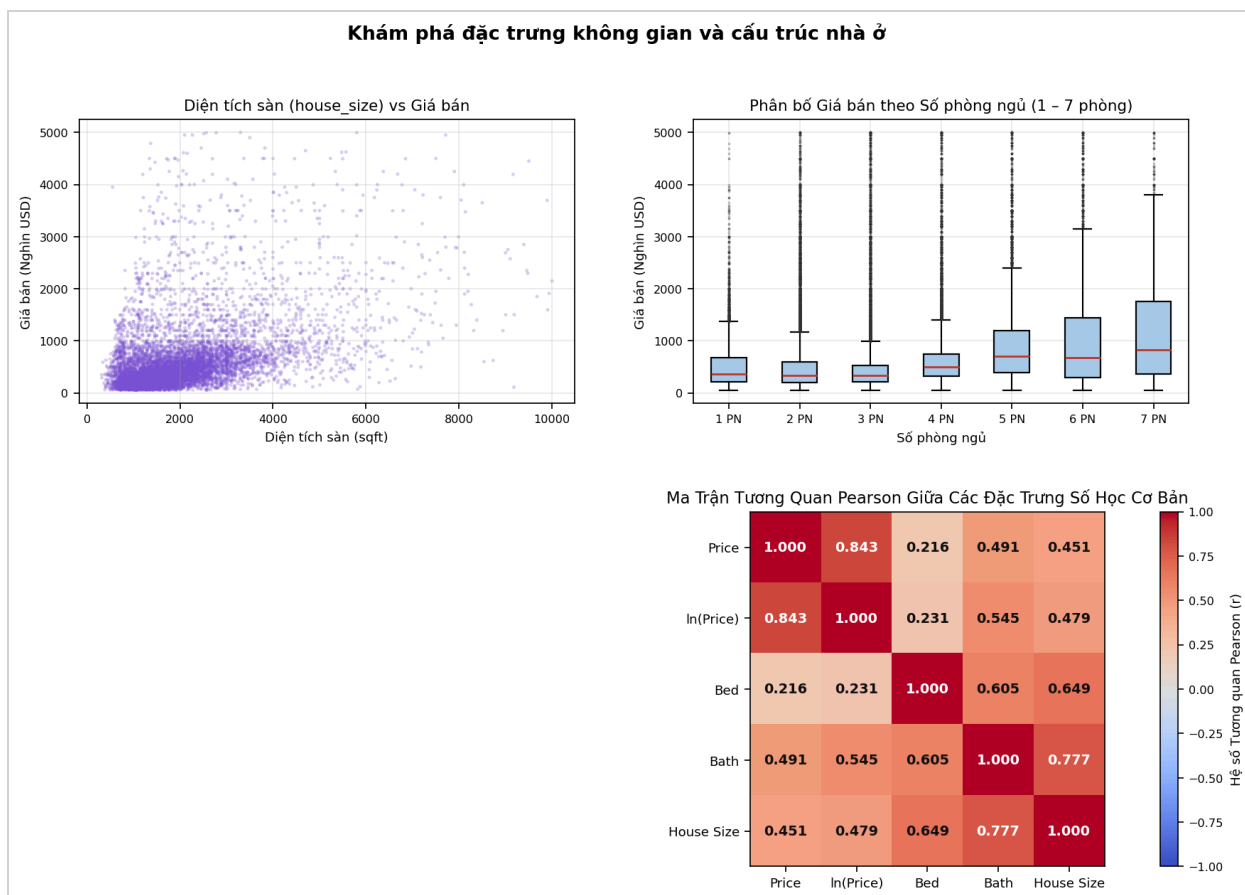
Áp dụng lý thuyết kinh tế lượng Hedonic, biến mục tiêu được chuyển đổi qua hàm logarit:

$$y_{log} = \log(1 + price)$$

Phép biến đổi đưa hệ số lệch từ **3.161** xuống **0.241** — gần như đối xứng chuẩn hoàn hảo. Nó cũng chuyển bài toán từ *sai số tuyệt đối* sang *sai số tương đối*, đúng với cách thị trường bất động sản vận hành: người ta nói "căn nhà này đắt hơn 20%", không nói "đắt hơn 80.000 USD".



Hình 12. Phân phối giá nhà thô (lệch phải nặng) và biến đổi Logarit đối xứng Gauss.



Hình 13. Khám phá đặc trưng không gian và cấu trúc nhà ở: quan hệ diện tích – giá, phân bố giá theo số phòng ngủ, và ma trận tương quan Pearson.

Quan sát. Đồ thị phân tán cho thấy quan hệ giữa diện tích sàn và giá bán mang hình dạng hình nón mở rộng chứ không phải một đường thẳng. Hệ số tương quan của diện tích với $\ln(\text{price})$ cao hơn với price thô.

Diễn giải. Quan hệ diện tích – giá mang bản chất **nhân tính** (giá tăng theo tỷ lệ phần trăm) chứ không phải cộng tính. Thêm 500 sqft cho một căn hộ nhỏ và cho một biệt thự tạo ra mức tăng giá tuyệt đối hoàn toàn khác nhau.

Ý nghĩa với học máy. Đây là bằng chứng định lượng ủng hộ quyết định biến đổi logarit ở trên: sau khi lấy log, quan hệ nhân tính trở thành quan hệ cộng tính, và mô hình tuyến tính mới có cơ hội khớp được dữ liệu.

4.2. Kiểm toán làm sạch dữ liệu và kỹ nghệ đặc trưng vị trí địa lý

Nhằm xây dựng tập dữ liệu huấn luyện tin cậy cho bài toán định giá quy mô lớn, một quy trình kiểm toán làm sạch được áp dụng tuần tự qua ba bước sàng lọc.

Bảng 18. Bảng kiểm toán quy trình làm sạch dữ liệu bất động sản (150,000 mẫu ban đầu).

Bước xử lý	Tiêu chí kiểm toán thực nghiệm	Số loại bỏ	Số còn lại	Tỷ lệ giữ
Dữ liệu thô ban đầu	Tập 150,000 tin đăng bất động sản toàn quốc	0	150,000	100.00%
1. Khuyết thiếu cốt lõi	Khuyết thiếu các trường buộc phải có: price, house_size, bed, bath, zip_code	8	149,992	99.99%
2. Khử trùng lặp	Trùng lặp hoàn toàn địa chỉ, giá và kết cấu công trình	158	149,834	99.89%
3. Lọc tin ảo & ngoại lai	Giá < \$50,000 hoặc > \$5,000,000; Diện tích < 300 hoặc > 10,000 sqft	2,301	147,533	98.36%

4.2.1. Kỹ nghệ đặc trưng hình học và tỷ lệ không gian

Dựa trên nguyên lý định giá bất động sản Hedonic, các đặc trưng tỷ lệ hình học và công năng được trích xuất nhằm tăng cường tín hiệu phi tuyến:

- **Xử lý khuyết thiếu diện tích lô đất (acre_lot):** trong tin đăng thực tế, nhiều căn hộ chung cư hoặc nhà phố liền kề không ghi nhận diện tích lô đất. Các giá trị khuyết thiếu này được điền bằng trung vị tính duy nhất trên tập huấn luyện (0.2300 acre).
- **Biến đổi logarit co giãn diện tích:** $\log_house_size = \ln(house_size)$ và $\log_acre_lot = \ln(1 + acre_lot)$.
- **Cấu trúc công năng xây dựng:** $total_rooms = bed + bath$, $sqft_per_room = house_size / total_rooms$, $bath_bed_ratio = bath / bed$, và $bed_bath_prod = bed \times bath$.

Chỉ số quy mô tương đối cục bộ (relative_sqft):

$$relative_sqft = house_size / mean(house_size | zip_code)$$

Đo lường mức độ bề thế **tương đối** của ngôi nhà so với mặt bằng quy mô dân cư lân cận cùng mã bưu chính. Một căn 2.000 sqft là bình thường ở ngoại ô nhưng là biệt thự ở trung tâm thành phố.

4.2.2. Mã hoá đích Bayes làm mượt (Smooth Bayesian Target Encoding) chống rò rỉ

Vị trí địa lý là nhân tố định đoạt giá trị bất động sản. Tuy nhiên, việc One-Hot Encoding hàng nghìn mã bưu chính sẽ làm bùng nổ chiều dữ liệu và gây thừa thớt nghiêm trọng. Để khắc phục, kỹ thuật Smooth Target Encoding có suy biến Bayes (Bayesian Shrinkage Smoothing) được triển khai cho bốn cấp độ địa lý:

$$S_i = (n_i \cdot \bar{y}_i + m \cdot \bar{y}_{global}) / (n_i + m)$$

Trong đó n_i và $\bar{y}_i$ là số lượng tin đăng và giá log trung bình của đơn vị địa lý i ; $\bar{y}_{global}$ là giá trị trung bình toàn cục trên tập huấn luyện; tham số làm mượt m đóng vai trò tiên nghiệm giả định (Prior Pseudo-counts) nhằm co kéo (shrink) các khu vực có quá ít giao dịch về mức giá trung bình toàn quốc, ngăn ngừa hiện tượng học vẹt cục bộ.

Bảng 19. Bốn cấp độ mã hoá đích Bayes với tham số làm mượt tương ứng.

Cấp độ địa lý	Tham số làm mượt m	Số đơn vị học được từ tập Train
Bang (state)	20	19
Vùng bưu chính 3 số (zip3)	15	223
Mã bưu chính 5 số (zip_code)	10	5,845
Thành phố (city)	10	4,594

Trực giác của công thức làm mượt. Một mã bưu chính chỉ có 2 tin đăng ($n_i = 2$) thì giá trung bình của nó gần như vô nghĩa về mặt thống kê — công thức sẽ kéo mạnh nó về giá trung bình toàn quốc. Ngược lại, một thành phố có 5.000 tin đăng ($n_i \gg m$) thì giá trung bình rất đáng tin — công thức giữ gần như nguyên giá trị đó.

Quy chuẩn chống rò rỉ dữ liệu. Toàn bộ bảng tra cứu ánh xạ S_i được tính **duy nhất trên tập Train** (118,026 mẫu) và ánh xạ cố định sang tập Test (29,507 mẫu). Các khu vực mới xuất hiện ở tập Test được gán giá trị trung bình toàn cục $\bar{y}_{global}$. Không gian đặc trưng sau cùng gồm đúng **13 biến đầu vào**, toàn bộ được chuẩn hoá Z-Score dựa trên thống kê của tập Train.

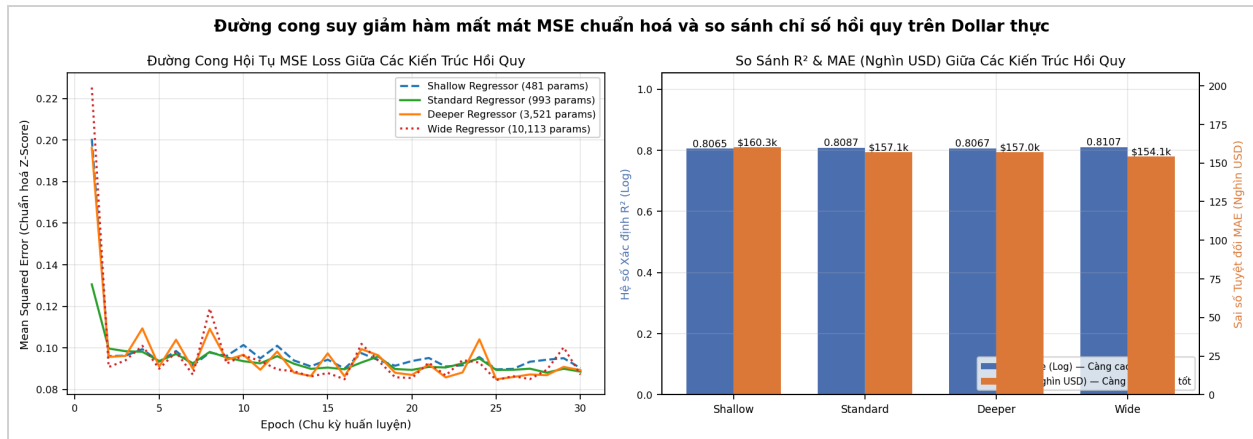
4.3. Khảo sát kiến trúc mạng hồi quy sâu: đột phá của chiều rộng

Để huấn luyện mạng hồi quy sâu thuần NumPy (`MLPRegressorScratch`), tầng ngõ ra sử dụng hàm kích hoạt tuyến tính $\phi(z) = z$ và hàm mất mát sai số toàn phương trung bình (MSE). Do $y_{\log}$ tiếp tục được chuẩn hoá Z-Score về phân phối có kỳ vọng bằng 0 và phương sai bằng 1, hàm mất mát huấn luyện là một đại lượng **không thứ nguyên** (Dimensionless), thuận tiện để so sánh giữa các kiến trúc.

Một quan sát toán học thú vị. Công thức gradient tầng ra của cặp *MSE + Linear* hoàn toàn giống với cặp *BCE + Sigmoid* ở Chương 3: $dZ^{[L]} = (\hat{y} - y) / m$. Đây không phải trùng hợp ngẫu nhiên — cả hai cặp đều thuộc họ hàm mất mát chính tắc (canonical link) của phân phối mũ tương ứng. Nhờ vậy toàn bộ phần lan truyền ngược qua các tầng ẩn giữ nguyên không đổi, chỉ cần thay hàm kích hoạt tầng cuối.

Bảng 20. Bảng đối chứng thực nghiệm 4 cấu hình kiến trúc mạng hồi quy sâu thuần NumPy.

Kiến trúc	Cấu hình tầng	Tham số	Thời gian	MSE Loss	R ² Score	MAE (USD)	RMSE (USD)
Shallow Regressor	13 → 32 → 1	481	2.28s	0.0903	0.8065	\$160,297	\$340,709
Standard Regressor	13 → 32 → 16 → 1	993	3.46s	0.0886	0.8087	\$157,093	\$328,776
Deeper Regressor	13 → 64 → 32 → 16 → 1	3,521	8.03s	0.0893	0.8067	\$157,003	\$313,676
Wide Regressor	13 → 128 → 64 → 1	10,113	20.08s	0.0871	0.8107	\$154,147	\$312,220



Hình 14. Đường cong suy giảm hàm mất mát MSE chuẩn hoá và so sánh chỉ số hồi quy trên Dollar thực.

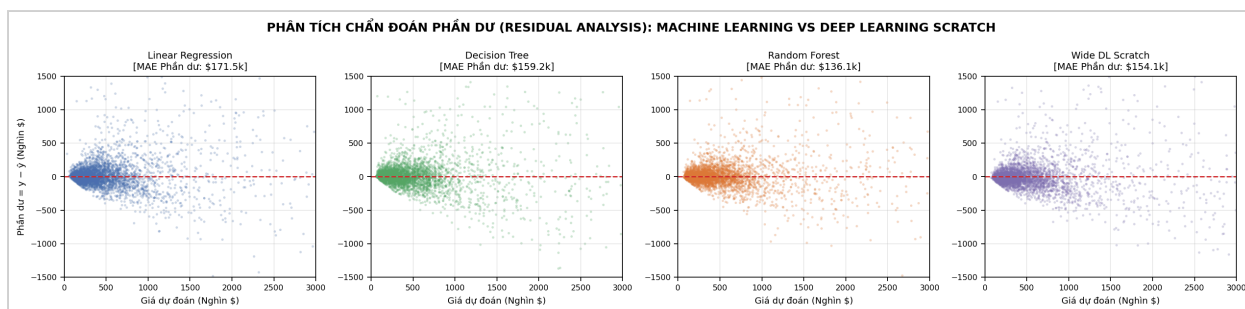
Quan sát. Khác với Chương 3, ở đây kiến trúc **Wide Regressor** ($13 \rightarrow 128 \rightarrow 64 \rightarrow 1$) mới là mô hình tốt nhất: MSE Loss thấp nhất 0.0871, R^2 cao nhất 0.8107 và MAE thấp nhất \$154,147. Mạng Deeper Regressor — ưu tiên chiều sâu — chỉ đạt R^2 0.8067.

Diễn giải. Bài toán phân loại cần **bề cong một siêu phẳng phân tách** để bao lấy các cụm dữ liệu, và việc bề cong phức tạp đòi hỏi tổ hợp phân cấp nhiều bậc — thế mạnh của chiều sâu. Bài toán hồi quy thì khác hẳn: nó cần **xấp xỉ một mặt phẳng giá trị liên tục, trơn và mấp mô** trải trên nhiều chiều đặc trưng.

Ý nghĩa với học máy. Mỗi nơ-ron ReLU tạo ra một "nếp gấp" tuyến tính từng đoạn (Piecewise Linear Approximator). Tăng chiều rộng tầng ẩn ($128 \rightarrow 64$) cung cấp một **hệ cơ sở hàm ReLU phong phú hơn**, cho phép ghép nối nhiều mặt phẳng con cục bộ mượt mà hơn — giống như dùng nhiều mảnh vải nhỏ để phủ một bề mặt cong thay vì ít mảnh lớn. **Quy luật kiến trúc của Chương 3 bị đảo ngược hoàn toàn.**

4.4. Phân tích chẩn đoán phần dư và sai số thị trường

Phần dư là hiệu giữa giá thực tế và giá dự đoán: $e_i = y_i - \hat{y}_i$. Đồ thị phần dư là công cụ chẩn đoán mạnh nhất của kinh tế lượng — nó tiết lộ những gì mà một con số R^2 đơn lẻ che giấu.



Hình 15. Chẩn đoán phần dư, kiểm tra phân phối chuẩn và phân tích sai số theo ba phân khúc thị trường.

Các phát hiện kinh tế lượng từ đồ thị phần dư:

- Phần dư đối xứng quanh 0.** Trung bình phần dư xấp xỉ \$3,273 trên mức giá trung vị \$399,900, tương đương tỷ lệ thiên lệch chỉ 0.82%. Điều này chứng minh mô hình **không bị thiên lệch hệ thống** (No Systematic Bias).
- Hiện tượng phương sai thay đổi (Heteroscedasticity).** Phễu phần dư mở rộng dần khi giá nhà dự đoán tăng lên. Điều này hoàn toàn phù hợp với thực tế kinh tế và được định lượng ở bảng dưới đây.

Bảng 21. Phân tích sai số định giá theo ba phân khúc thị trường bất động sản.

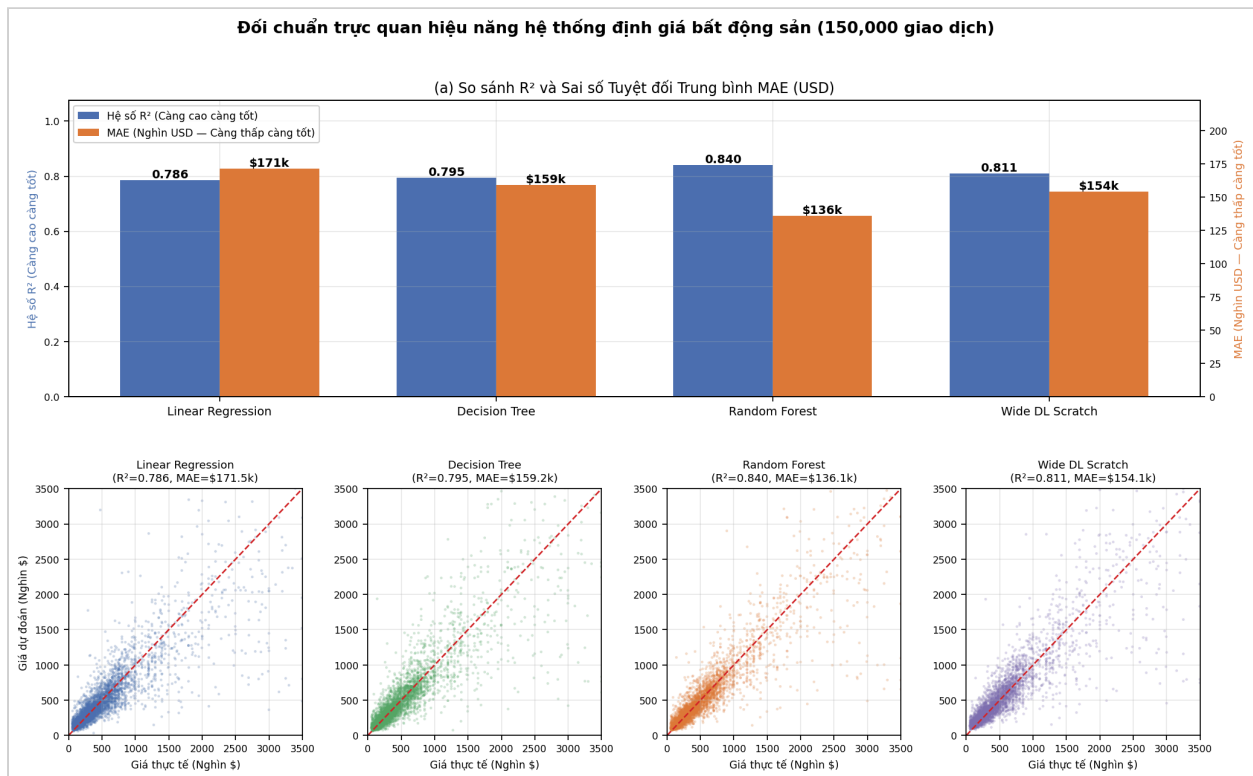
Phân khúc thị trường	Số giao dịch	Tỷ trọng	MAE	MAPE
Phổ thông (< \$300k)	10,657	36.1%	\$67,427	45.6%
Trung cấp (\$300k – \$800k)	13,274	45.0%	\$106,541	21.2%
Cao cấp & Biệt thự (> \$800k)	5,576	18.9%	\$433,219	25.4%

Ở phân khúc **Trung cấp (\$300k – \$800k)** — chiếm 45.0% thị trường — mô hình định giá chuẩn xác nhất với MAPE chỉ 21.2%. Ở phân khúc **Cao cấp & Biệt thự (> \$800k)**, sai số tuyệt đối vọt lên \$433,219: giá cả phụ thuộc nhiều vào yếu tố độc bản (nội thất xa xỉ, tầm nhìn cảnh quan, uy tín kiến trúc sư) mà dữ liệu thuộc tính thông thường không nắm bắt hết được. Đáng chú ý, phân khúc **Phổ thông (< \$300k)** có MAE thấp nhất (\$67,427) nhưng MAPE lại cao nhất (45.6%) — vì mẫu số nhỏ khiến cùng một sai số tuyệt đối trở thành tỷ lệ phần trăm lớn.

4.5. Tổng hợp đối chuẩn 4 mô hình định giá bất động sản

Bảng 22. Bảng đối chuẩn toàn diện 4 mô hình định giá nhà trên không gian giá trị Dollar thực.

Mô hình	Loại mô hình	Tham số	Thời gian	R ² (Log)	R ² (Raw)	MAE (USD)	MAPE
Linear Regression	Tuyến tính OLS	14	0.049s	0.7860	0.6556	171,493.89	31.95%
Decision Tree	Cây hồi quy	12,309	1.109s	0.7953	0.7654	159,241.09	30.06%
Random Forest	Ensemble 100 Cây	3,548,570	9.842s	0.8404	0.8134	136,124.25	25.72%
Wide DL Scratch	Mạng nơ-ron rộng (13 → 128 → 64 → 1)	10,113	20.079s	0.8107	0.7724	154,147.12	30.78%



Hình 16. Đối chuẩn trực quan hiệu năng hệ thống định giá bất động sản (150,000 giao dịch).

4.5.1. Nhận xét chuyên sâu và đánh giá kết quả thực nghiệm định giá nhà

1. Sự phân kỳ giữa không gian Logarit và không gian Dollar thực.

Linear Regression đạt $R^2(\log) = 0.7860$ — nghe khá tốt — nhưng $R^2(\text{raw})$ tụt xuống chỉ còn **0.6556**, và sai số MAE vọt lên \$171,493.89. Nguyên nhân nằm ở bản chất hàm mũ.

Nếu $\log(y) = w^T x + b + \varepsilon$ thì khi nghịch đảo:

$$y = \exp(w^T x + b) \cdot \exp(\varepsilon) \approx \exp(w^T x + b) \cdot (1 + \varepsilon + \varepsilon^2/2)$$

Một sai số dư nhỏ $\varepsilon = 0,30$ trên thang log của căn nhà trị giá 3.000.000 USD sẽ bị khuếch đại thành mức chênh lệch vượt quá 1.050.000 USD trong giá trị thực. **Bài học: phải luôn báo cáo song song cả hai chỉ số R^2 .**

2. Ưu thế của mạng rộng trong xấp xỉ mặt phẳng liên tục.

Mô hình Wide Regressor ($13 \rightarrow 128 \rightarrow 64 \rightarrow 1$, 10,113 tham số) đạt $R^2(\log) = 0.8107$, $MAE = \$154,147.12$ và $MAPE = 30.78\%$. Mô hình vượt Linear Regression tới **\$17,347 sai số trên mỗi căn nhà**. 128 nơ-ron tại tầng ẩn thứ nhất cung cấp một hệ cơ sở hàm ReLU phong phú, giúp mô hình bẻ gập và khớp nối các bề mặt giá trị đa chiều mấp mô một cách trơn tru.

3. Random Forest và bản chất phân mảnh của thị trường bất động sản.

Random Forest giành vị trí dẫn đầu với $R^2(\text{raw}) = 0.8134$, $MAE = \$136,124.25$ và $MAPE = 25.72\%$. Thị trường bất động sản không phải một mặt phẳng liên tục toàn cục mà là tập hợp của hàng nghìn thị trường con địa phương với quy luật giá riêng biệt. Các bề mặt xấp xỉ từng đoạn cục bộ (Piecewise Approximation) của rừng cây kìm hãm sự bùng nổ sai số ở đuôi giá cao — đúng nơi mà mô hình tuyến tính sụp đổ ($R^2(\text{raw})$ chỉ 0.6556).

Chương 5. Hệ thống 3 — Phân loại nhận xét thương mại điện tử (22,636 đánh giá NLP)

Hai chương trước làm việc trên **dữ liệu bảng có cấu trúc**: mỗi cột là một đại lượng đo được, mỗi hàng là một quan sát. Chương này bước sang miền dữ liệu **phi cấu trúc**: văn bản tự do do khách hàng viết ra.

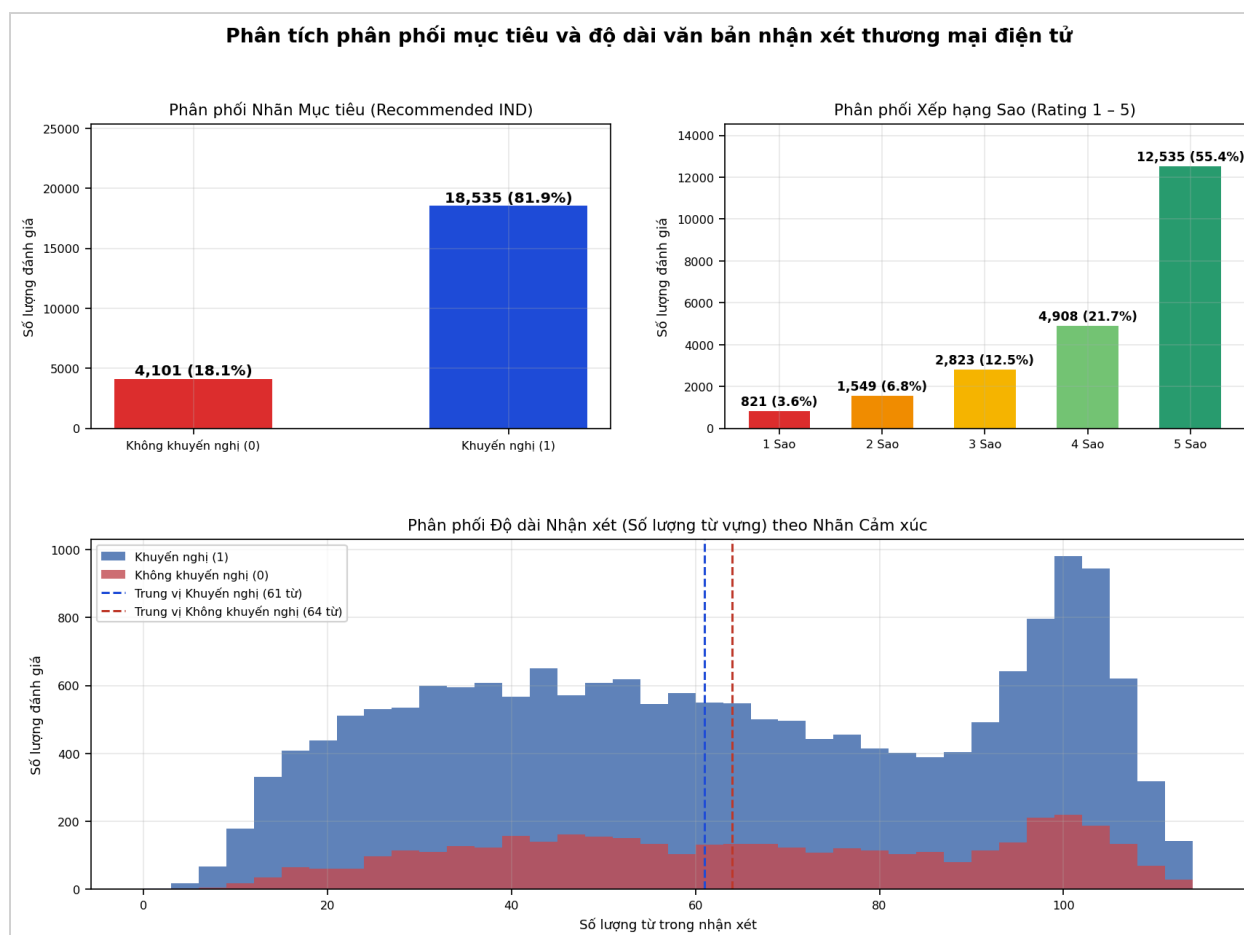
Sự chuyển đổi này đặt ra một thách thức hoàn toàn mới. Một mạng nơ-ron chỉ nhận vào các vector số thực. Vậy làm thế nào biến câu "chiếc váy này rất đẹp nhưng size hơi nhỏ" thành một vector? Đó chính là bài toán **biểu diễn văn bản** (Text Representation) mà chương này giải quyết bằng TF-IDF.

Bảng 23. Đối chiếu bản chất dữ liệu bảng và dữ liệu văn bản phi cấu trúc.

Đặc điểm	Dữ liệu bảng (Chương 3, 4)	Dữ liệu văn bản (Chương 5)
Nguồn đặc trưng	Đo lường trực tiếp (tuổi, BMI, diện tích)	Trích xuất từ chuỗi ký tự
Số chiều	13 – 14 chiều	1,000 chiều
Mật độ ma trận	Đặc (dense) gần 100%	Thưa 97.8% giá trị bằng 0
Ý nghĩa mỗi chiều	Một đại lượng vật lý/sinh học	Trọng số của một từ hoặc cụm hai từ

5.1. Bối cảnh văn bản e-commerce và không gian TF-IDF

Tập dữ liệu `Women's Clothing E-Commerce Reviews` gồm 23,486 đánh giá sản phẩm thời trang nữ. Mỗi bản ghi chứa tiêu đề, nội dung nhận xét, điểm đánh giá sao và nhãn nhị phân `Recommended IND` — biến mục tiêu cần dự đoán: khách hàng có giới thiệu sản phẩm này cho người khác hay không.

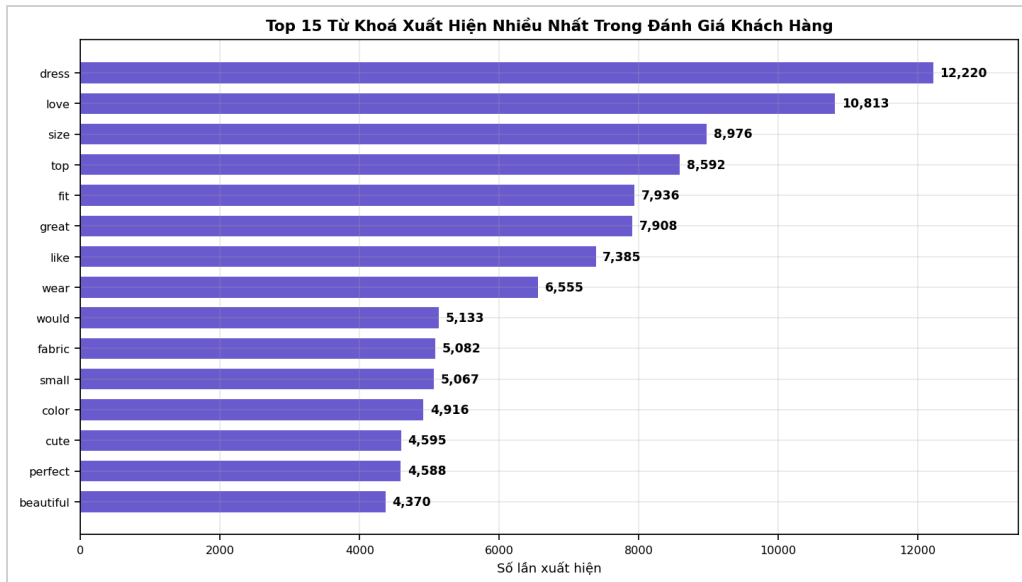


Hình 17. Phân tích phân phối mục tiêu và độ dài văn bản nhận xét thương mại điện tử.

Quan sát. Tỷ lệ nhấn tích cực chiếm **81.9%** — mất cân bằng theo hướng ngược lại so với Chương 3. Độ dài nhận xét có trung vị 61 từ ở nhóm tích cực và 64 từ ở nhóm tiêu cực.

Diễn giải. Khách hàng không hài lòng viết dài hơn khách hàng hài lòng. Người hài lòng thường chỉ cần vài từ ("love it", "perfect fit"), trong khi người thất vọng có xu hướng giải thích cặn kẽ lý do trả hàng.

Ý nghĩa với học máy. Chênh lệch độ dài này là một tín hiệu phân biệt tiềm năng, nhưng nó *không* được đưa vào mô hình như một đặc trưng riêng — mục tiêu của chương là kiểm tra xem mô hình học được gì **chỉ từ nội dung ngữ nghĩa** của văn bản.



Hình 18. Top 15 từ khoá xuất hiện nhiều nhất trong đánh giá của khách hàng.

Tám từ khoá xuất hiện dày đặc nhất là **dress**, **love**, **size**, **top**, **fit**, **great**, **like**, **wear**. Đây chủ yếu là danh từ chỉ sản phẩm và tính từ đánh giá — đúng như kỳ vọng với một tập dữ liệu nhận xét thời trang.

5.1.1. Quy trình làm sạch dữ liệu văn bản (NLP Text Cleaning)

Bảng 24. Bảng kiểm toán quy trình làm sạch dữ liệu văn bản NLP thương mại điện tử (23,486 nhận xét ban đầu).

Bước xử lý	Tiêu chí tiền xử lý văn bản	Số loại bỏ	Số còn lại	Tỷ lệ giữ
Dữ liệu thô ban đầu	Tập đánh giá sản phẩm thương mại điện tử gốc	0	23,486	100.00%
1. Lọc khuyết thiếu nội dung	Loại bỏ bản ghi khuyết trường Review Text hoặc nhãn Recommended IND	845	22,641	96.40%
2. Hợp nhất ngữ cảnh	Ghép nối tiêu đề Title và nội dung Review Text thành văn bản đầy đủ	0	22,641	96.40%
3. Khử trùng lặp văn bản	Loại bỏ các đánh giá có nội dung văn bản hoàn toàn trùng khớp	5	22,636	96.38%

5.1.2. Phân chia phân tầng và trích xuất TF-IDF chống rò rỉ

TF-IDF (Term Frequency – Inverse Document Frequency) gán cho mỗi từ một trọng số phản ánh mức độ *đặc trưng* của nó với một văn bản cụ thể:

$$tfidf(t, d) = tf(t, d) \times \log(N / df(t))$$

Một từ xuất hiện nhiều trong một nhận xét (tf cao) nhưng hiếm gặp trong toàn bộ tập dữ liệu (df thấp) sẽ nhận trọng số lớn — đó là từ mang tính phân biệt. Ngược lại, từ xuất hiện ở mọi văn bản (như "the", "and") nhận trọng số gần 0.

Bảng 25. Cấu hình bộ vector hoá TF-IDF.

Tham số cấu hình	Giá trị	Lý do lựa chọn
<code>max_features</code>	1,000	Giữ 1.000 từ/cụm từ có trọng số cao nhất, cân bằng biểu diễn và chi phí tính toán
<code>ngram_range</code>	(1, 2)	Bắt cả từ đơn lẫn cụm hai từ — 224 bigram được giữ lại, nắm được cụm phủ định như "not worth"
<code>stop_words</code>	english	Loại bỏ hư từ tiếng Anh không mang thông tin phân loại
<code>sublinear_tf</code>	True	Dùng $1 + \log(tf)$ thay tf thô, giảm ảnh hưởng của từ lặp quá nhiều lần

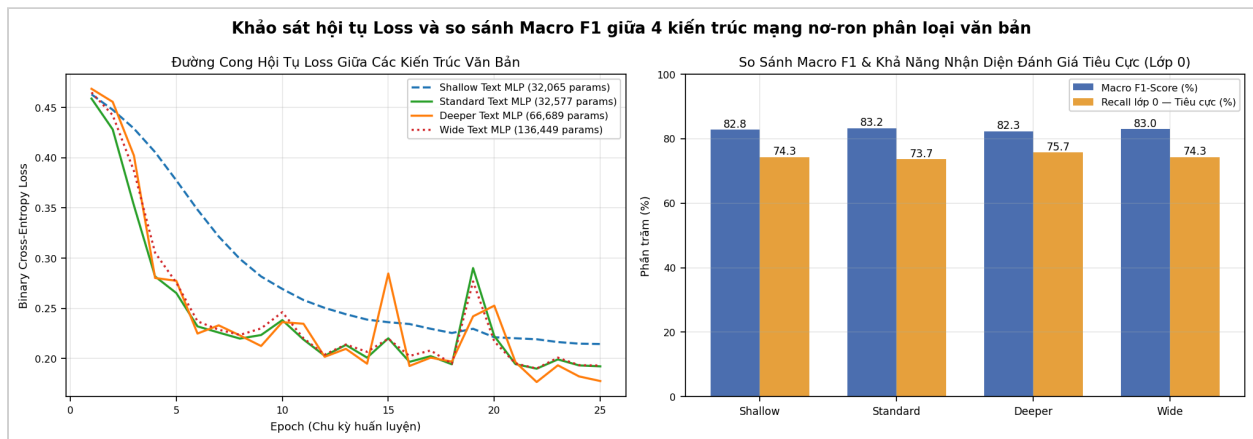
Bộ vector hoá được `fit` duy nhất trên tập **Train** (18,109 mẫu) rồi `transform` sang tập **Test** (4,527 mẫu). Nếu học từ điển trên toàn bộ dữ liệu, các từ chỉ xuất hiện ở tập **Test** sẽ tham gia vào từ điển — một dạng rò rỉ dữ liệu. Ma trận kết quả có độ thưa **97.81%**: trung bình mỗi nhận xét chỉ kích hoạt khoảng 22 trong số 1,000 chiều.

5.2. Khảo sát kiến trúc NLP: thích ứng trên dữ liệu thưa chiều cao

Bốn kiến trúc mạng được huấn luyện trên không gian TF-IDF 1,000 chiều với tốc độ học $\eta = 0.1$, kích thước lô 128 và 25 epochs.

Bảng 26. Bảng đối chứng thực nghiệm khảo sát kiến trúc mạng văn bản thuần NumPy.

Kiến trúc	Cấu hình tầng	Tham số	Thời gian	Loss cuối	Accuracy	Recall lớp tiêu cực	Macro F1
Shallow Text MLP	1000 → 32 → 1	32,065	2.17s	0.2145	89.57%	74.27%	82.83%
Standard Text MLP	1000 → 32 → 16 → 1	32,577	2.28s	0.1922	89.95%	73.66%	83.24%
Deeper Text MLP	1000 → 64 → 32 → 16 → 1	66,689	3.67s	0.1776	89.04%	75.73%	82.34%
Wide Text MLP	1000 → 128 → 64 → 1	136,449	5.59s	0.1930	89.71%	74.27%	83.00%



Hình 19. Khảo sát hội tụ Loss và so sánh Macro F1 giữa 4 kiến trúc mạng nơ-ron phân loại văn bản.

Quan sát. Kiến trúc **Standard Text MLP** (1000 → 32 → 16 → 1, 32,577 tham số) đạt Macro F1 cao nhất 83.24%. Mạng Wide Text MLP có tới 136,449 tham số — gấp 4.2 lần — nhưng Macro F1 chỉ 83.00%.

Diễn giải. Trên không gian thưa chiều cao, phần lớn tham số của tầng đầu tiên không bao giờ được cập nhật trong một lô dữ liệu bất kỳ: với độ thưa 97.8%, mỗi mẫu chỉ kích hoạt vài chục chiều đầu vào. Tăng chiều rộng chỉ làm tăng số tham số "chết" và tạo cơ hội học vẹt các từ hiếm.

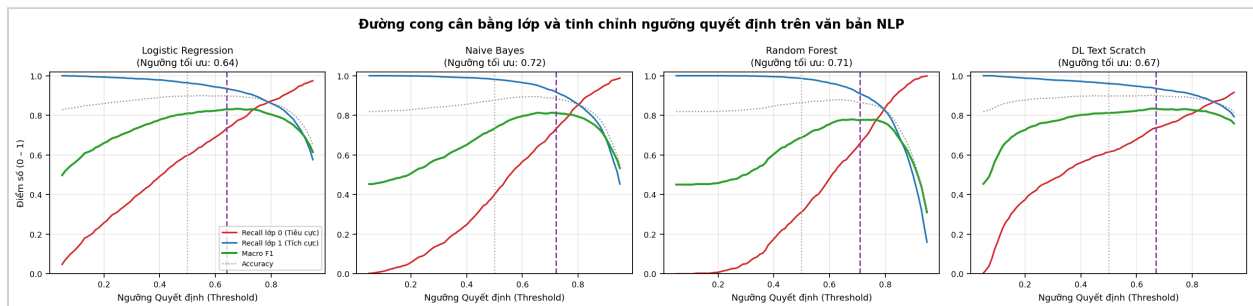
Ý nghĩa với học máy. Bài toán văn bản có quy luật kiến trúc riêng, khác cả Chương 3 lẫn Chương 4: **không phải sâu hơn, cũng không phải rộng hơn, mà là vừa đủ.** Điều quyết định chất lượng ở đây là chất lượng phép biểu diễn TF-IDF, không phải độ phức tạp của mạng.

5.3. Tinh chỉnh ngưỡng cân bằng lớp (Macro F1 Optimization)

Với 81.9% nhãn tích cực, việc tối ưu theo F1 của riêng lớp đa số sẽ che giấu hoàn toàn năng lực phát hiện nhận xét tiêu cực — vốn mới là thứ doanh nghiệp cần. Vì vậy chương này chuyển sang tối ưu **Macro F1**:

$$Macro\ F1 = (F1_{lớp\ 0} + F1_{lớp\ 1}) / 2$$

Chỉ số này đối xử với hai lớp *bình đẳng* bất kể số lượng mẫu, buộc mô hình phải làm tốt ở cả hai phía.



Hình 20. Đường cong cân bằng lớp và tinh chỉnh ngưỡng quyết định trên văn bản NLP.

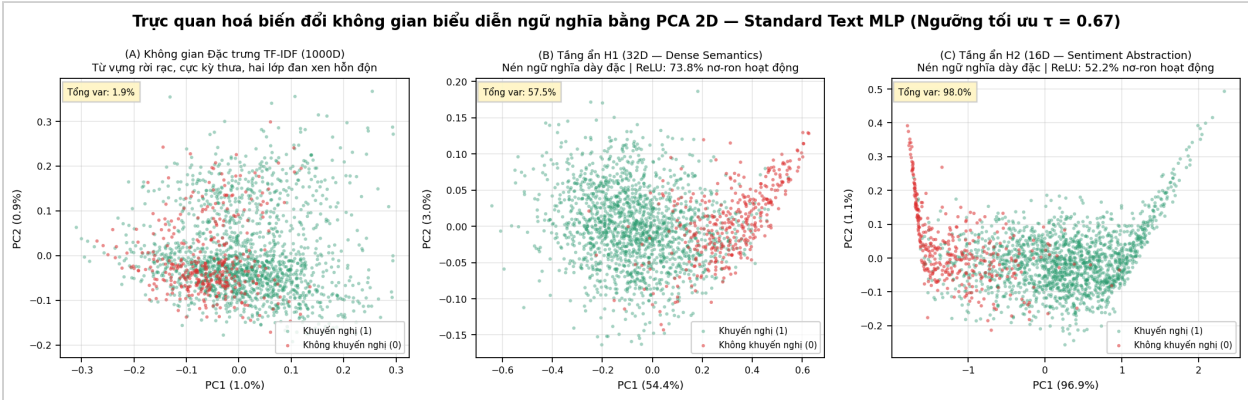
Bảng 27. Hiệu quả tinh chỉnh ngưỡng lên năng lực phát hiện nhận xét tiêu cực.

Mô hình	Ngưỡng τ	Macro F1 (0,50)	Macro F1 (Opt)	Recall lớp 0 (0,50)	Recall lớp 0 (Opt)	Δ Recall lớp 0
Logistic Regression	0.64	81.01%	83.05%	59.76%	73.29%	+13.54%
Naive Bayes	0.72	73.41%	81.13%	40.12%	73.05%	+32.93%
Random Forest	0.71	68.84%	77.65%	31.22%	65.98%	+34.76%
DL Text Scratch	0.67	81.16%	83.24%	61.34%	73.66%	+12.32%

Điểm chung của cả bốn mô hình: ngưỡng tối ưu đều nằm **cao hơn** 0,50 (ngược hẳn với Chương 3). Nguyên nhân là hướng mất cân bằng đảo chiều — ở đây lớp đa số là lớp tích cực, nên xác suất dự đoán bị dồn về phía 1, và phải nâng ngưỡng lên mới phát hiện được nhận xét tiêu cực.

5.4. Trực quan hoá quá trình học biểu diễn ngữ nghĩa bằng PCA 2D

Không gian TF-IDF ban đầu có 1,000 chiều với độ thưa 97.8%. Câu hỏi đặt ra: mạng nơ-ron biến đổi không gian thưa thớt này thành cái gì ở các tầng ẩn?



Hình 21. Trực quan hoá biến đổi không gian biểu diễn ngữ nghĩa bằng PCA 2D.

Quan sát. Ở không gian TF-IDF đầu vào, hai lớp nhận xét chồng lẫn gần như hoàn toàn — điều dễ hiểu vì phần lớn từ vựng (tên sản phẩm, chất liệu, màu sắc) xuất hiện ở cả nhận xét tích cực lẫn tiêu cực. Qua các tầng ẩn, cụm màu đỏ (không giới thiệu) tách dần ra một phía.

Diễn giải. Mạng đã học được cách **lọc bỏ từ vựng trung tính** và khuếch đại trọng số của các từ mang cảm xúc. Phép chiếu từ 1.000 chiều thưa xuống 32 chiều đặc chính là một dạng nén ngữ nghĩa (Semantic Compression) do mạng tự khám phá.

Ý nghĩa với học máy. Đây là bằng chứng cho thấy Representation Learning hoạt động không chỉ trên dữ liệu số mà cả trên văn bản: mạng tự tìm ra một không gian ngữ nghĩa nơi "khen" và "chê" nằm ở hai vùng tách biệt, dù chưa từng được dạy khái niệm cảm xúc nào.

Bảng 28. Các từ khoá mang trọng số phân cực mạnh nhất do mô hình tuyến tính học được.

Hạng	Từ khoá tiêu cực	Trọng số	Từ khoá tích cực	Trọng số
1	disappointed	-5.642	love	+5.921
2	wanted love	-4.737	perfect	+4.483
3	unflattering	-4.249	great	+4.069
4	cheap	-3.880	unique	+3.626
5	poor	-3.842	comfortable	+3.550

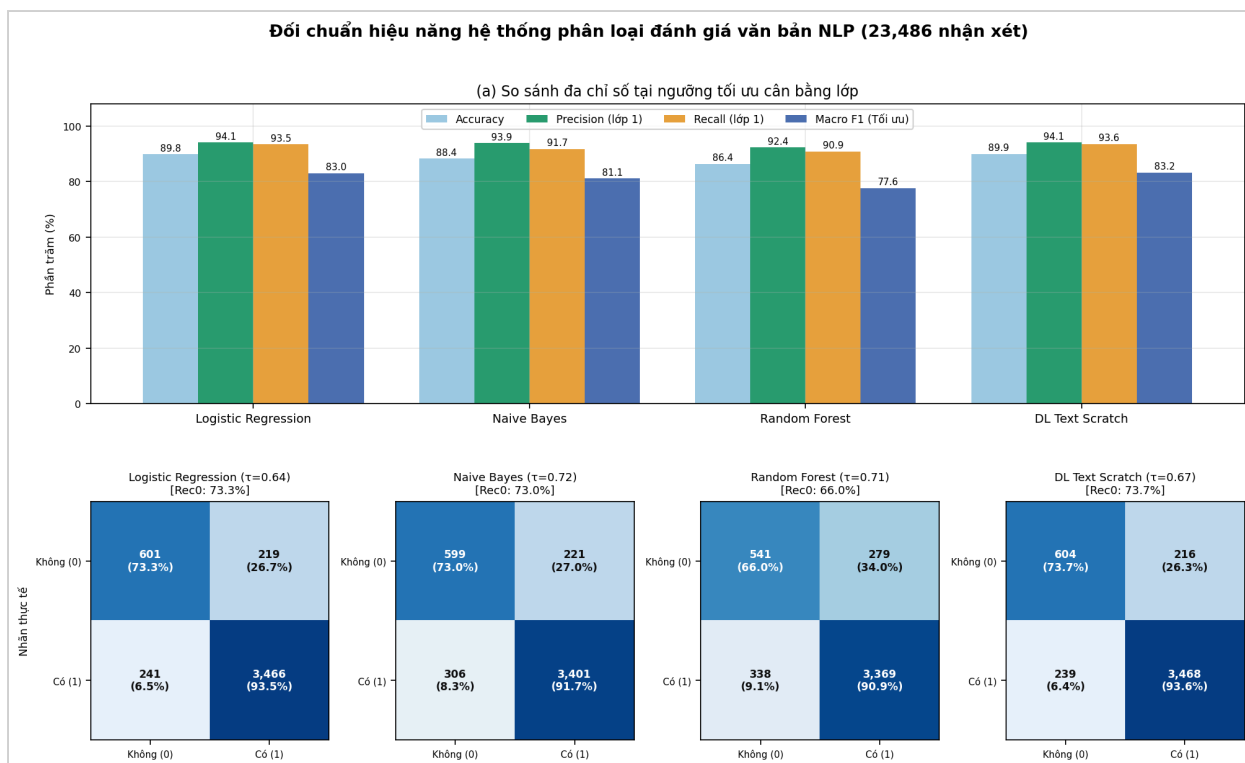
Hạng	Từ khoá tiêu cực	Trọng số	Từ khoá tích cực	Trọng số
6	returned	-3.531	soft	+3.503
7	huge	-3.463	compliments	+3.362
8	disappointing	-3.334	amazing	+3.306

Bảng trọng số này là minh chứng cho thấy mô hình học được ngữ nghĩa thật chứ không phải nhiễu ngẫu nhiên. Đặc biệt đáng chú ý là cụm bigram `wanted love` nằm trong nhóm tiêu cực mạnh nhất — cụm này đến từ mẫu câu "I wanted to love this but...", một cách diễn đạt thất vọng lịch sự mà mô hình chỉ có thể bắt được nhờ cấu hình `ngram_range = (1, 2)`. Nếu chỉ dùng từ đơn, cả hai từ *wanted* và *love* đều mang sắc thái trung tính hoặc tích cực.

5.5. Tổng hợp đối chuẩn 4 mô hình xử lý văn bản NLP

Bảng 29. Bảng đối chuẩn khoa học: Machine Learning và Deep Learning thuần NumPy (22,636 nhận xét).

Mô hình	Loại mô hình	Tham số	Thời gian	Ngưỡng	Accuracy	Macro F1	Recall lớp 0	ROC-AUC
Logistic Regression	ML Tuyến tính	1,001	0.050s	0.64	89.84%	83.05%	73.29%	0.9348
Naive Bayes	Xác suất Bayes	2,000	0.003s	0.72	88.36%	81.13%	73.05%	0.9276
Random Forest	Ensemble 50 Cây	51,050	0.252s	0.71	86.37%	77.65%	65.98%	0.9047
DL Text Scratch	Mạng 2 lớp rộng	32,577	2.281s	0.67	89.95%	83.24%	73.66%	0.9332



Hình 22. Đổi chuẩn trực quan hiệu năng hệ thống phân loại đánh giá văn bản NLP.

5.5.1. Nhận xét chuyên sâu và đánh giá kết quả phân loại văn bản NLP

1. Biểu diễn quan trọng hơn thuật toán.

Bốn mô hình có độ phức tạp chênh nhau rất xa — từ Naive Bayes với 2,000 tham số đến mạng nơ-ron 32,577 tham số — nhưng Macro F1 chỉ dao động trong khoảng 77.65% – 83.24%. Khoảng cách 5.59 điểm là rất nhỏ so với khoảng cách giữa các mô hình ở Chương 3 và 4. **Chất lượng phép biểu diễn TF-IDF mới là yếu tố quyết định**, không phải độ tinh vi của thuật toán phân loại phía sau.

2. Naive Bayes — mô hình đơn giản nhất, tốc độ vượt trội.

Naive Bayes huấn luyện trong 0.0030 giây — nhanh gấp 769 lần mạng nơ-ron — mà vẫn đạt Macro F1 81.13% và ROC-AUC 0.9276. Giả định độc lập điều kiện giữa các từ (naive) tuy sai về mặt ngôn ngữ học nhưng lại rất hiệu quả trên ma trận đếm từ thưa. Trong một hệ thống thực tế cần xử lý hàng triệu nhận xét mỗi ngày, đây là đánh đổi rất đáng cân nhắc.

3. Random Forest và giới hạn của cây quyết định trên dữ liệu thưa.

Random Forest — mô hình thống trị ở Chương 3 và 4 — lại đứng cuối bảng ở đây với ROC-AUC 0.9047, thấp hơn Logistic Regression (0.9348). Cây quyết định chọn *một* đặc trưng tại mỗi nút để chia; trên không gian 1,000 chiều với độ thưa 97.8%, mỗi phép chia

chỉ dựa trên sự có mặt của một từ đơn lẻ. Trong khi đó, cảm xúc của một câu là **tổng hợp tuyến tính** của nhiều từ cùng lúc — đúng là thứ mà Logistic Regression và mạng nơ-ron làm tự nhiên.

4. Mạng nơ-ron thuần NumPy trên không gian 1.000 chiều.

Mô hình Standard Text MLP ($1000 \rightarrow 32 \rightarrow 16 \rightarrow 1$) đạt Macro F1 83.24% và Recall lớp tiêu cực 73.66% — dẫn đầu bảng đối chuẩn. Điều đáng ghi nhận là toàn bộ 32,577 tham số này được huấn luyện bằng các phương trình lan truyền ngược viết tay, hội tụ ổn định chỉ trong 2.28 giây trên 18,109 mẫu văn bản.

Chương 6. Tổng hợp đối chuẩn khoa học đa chiều và thảo luận chuyên sâu

Ba chương trước đã giải quyết ba bài toán thuộc ba họ khác nhau — phân loại nhị phân mất cân bằng, hồi quy giá trị liên tục, và phân loại văn bản phi cấu trúc. Chương này đặt toàn bộ **12 mô hình** lên cùng một mặt bàn để rút ra các quy luật xuyên suốt.

6.1. Bảng tổng hợp đối chuẩn 12 mô hình trên cả ba hệ thống

Bảng 30. Bảng đối chuẩn tổng hợp đa chiều 12 mô hình trên ba bài toán hệ thống thông minh quy mô lớn. HT1 = sàng lọc tiểu đường, HT2 = định giá bất động sản, HT3 = phân loại nhận xét NLP.

HT	Mô hình / Loại	Tham số	Thời gian	Chỉ số chính	Chỉ số phụ
HT1	Logistic Regression Tuyến tính	15	0.260s	F1 73.20%	AUC 0.9582 Recall 64.50%
HT1	Decision Tree Cây đơn lẻ	335	0.306s	F1 80.55%	AUC 0.9709 Recall 71.40%
HT1	Random Forest Ensemble 100 Cây	122,538	0.784s	F1 79.64%	AUC 0.9754 Recall 73.11%
HT1	Deeper DL Scratch Deep MLP (14 → 64 → 32 → 16 → 1)	3,585	5.703s	F1 78.35%	AUC 0.9716 Recall 67.75%
HT2	Linear Regression Tuyến tính OLS	14	0.049s	R ² 0.6556	MAE \$171,494 MAPE 31.95%
HT2	Decision Tree Cây hồi quy	12,309	1.109s	R ² 0.7654	MAE \$159,241 MAPE 30.06%
HT2	Random Forest Ensemble 100 Cây	3,548,570	9.842s	R ² 0.8134	MAE \$136,124 MAPE 25.72%
HT2	Wide DL Scratch Mạng nơ-ron rộng (13 → 128 → 64 → 1)	10,113	20.079s	R ² 0.7724	MAE \$154,147 MAPE 30.78%
HT3	Logistic Regression ML Tuyến tính	1,001	0.050s	Macro F1 83.05%	AUC 0.9348 Reco 73.29%

HT	Mô hình / Loại	Tham số	Thời gian	Chỉ số chính	Chỉ số phụ
HT3	Naive Bayes Xác suất Bayes	2,000	0.003s	Macro F1 81.13%	AUC 0.9276 Reco 73.05%
HT3	Random Forest Ensemble 50 Cây	51,050	0.252s	Macro F1 77.65%	AUC 0.9047 Reco 65.98%
HT3	DL Text Scratch Mạng 2 lớp rộng	32,577	2.281s	Macro F1 83.24%	AUC 0.9332 Reco 73.66%

Vì sao mỗi hệ thống dùng một chỉ số chính khác nhau? Không có một thước đo duy nhất áp dụng được cho cả ba bài toán. Hệ thống 1 mất cân bằng nặng nên Accuracy vô nghĩa, phải dùng F1 và Recall. Hệ thống 2 là hồi quy nên không tồn tại khái niệm Accuracy, phải dùng R^2 và MAE. Hệ thống 3 mất cân bằng theo chiều ngược lại nên phải dùng Macro F1 để hai lớp được đối xử bình đẳng. **Chọn sai chỉ số là cách nhanh nhất để rút ra kết luận sai.**

6.2. Thảo luận về kiến trúc và triết lý lựa chọn mô hình

6.2.1. Không tồn tại một kiến trúc tối ưu cho mọi bài toán

Bảng 31. Ba bài toán, ba quy luật kiến trúc hoàn toàn khác nhau.

Hệ thống	Bài toán	Kiến trúc thắng cuộc	Cấu hình tầng	Quy luật rút ra
HT1	Phân loại nhị phân	Deeper MLP	14 → 64 → 32 → 16 → 1	Chiều sâu thắng — cần tổ hợp phân cấp để bề cong siêu phẳng
HT2	Hồi quy liên tục	Wide Regressor	13 → 128 → 64 → 1	Chiều rộng thắng — cần nhiều bộ xấp xỉ tuyến tính từng đoạn
HT3	Phân loại văn bản	Standard Text MLP	1000 → 32 → 16 → 1	Vừa đủ thắng — dữ liệu thừa, thêm tham số chỉ gây học vẹt

Đây là phát hiện quan trọng nhất về mặt kiến trúc của toàn bộ nghiên cứu. Cùng một lớp mạng `MLPScratch`, cùng cách khởi tạo He Normal, cùng thuật toán tối ưu Mini-Batch Gradient Descent — nhưng cấu hình tầng tối ưu lại khác nhau ở cả ba bài toán, và khác theo hướng có thể giải thích được:

- **Phân loại** đòi hỏi mô hình *bể cong một mặt phân tách*. Mỗi tầng ẩn bổ sung cho phép gấp không gian thêm một lần, nên chiều sâu tạo ra năng lực biểu diễn theo cấp số nhân.
- **Hồi quy** đòi hỏi mô hình *phủ một mặt giá trị liên tục*. Mỗi nơ-ron ReLU là một mảnh phẳng; càng nhiều mảnh song song thì bề mặt càng mượt, nên chiều rộng mới là thứ quyết định.
- **Văn bản thưa chiều cao** đặt ra ràng buộc khác hẳn: phần lớn tham số tầng đầu không được cập nhật trong mỗi lô (độ thưa 97.8%), nên tăng kích thước mạng chỉ làm tăng số tham số chết.

6.2.2. Khi nào nên chọn Machine Learning, khi nào nên chọn Deep Learning?

Bảng 32. Mô hình thắng cuộc ở mỗi hệ thống và nguyên nhân cấu trúc đằng sau.

Hệ thống	Quán quân	Vì sao thắng
HT1 — Tiểu đường	Decision Tree	Ranh giới bệnh được xác lập bởi ngưỡng sinh hoá nghiêm ngặt ($HbA1c \geq 6,5\%$). Lát cắt trực giao của cây bắt trọn bước nhảy bậc thang.
HT2 — Bất động sản	Random Forest	Thị trường là tập hợp hàng nghìn thị trường con địa phương . Xấp xỉ từng đoạn cục bộ của rừng cây kìm hãm bùng nổ sai số ở đuôi giá cao.
HT3 — Nhận xét NLP	DL Text Scratch	Cảm xúc câu là tổng hợp tuyến tính của nhiều từ cùng lúc — đúng thứ mạng nơ-ron và mô hình tuyến tính làm tự nhiên, còn cây thì không.

Ba kết quả này gộp lại cho một nguyên tắc chọn mô hình có thể áp dụng ngay:

- **Dữ liệu bảng có ngưỡng rõ ràng** (y tế, tài chính, kiểm định chất lượng) → ưu tiên mô hình cây. Chúng nhanh hơn, dễ giải thích hơn, và thường thắng.
- **Dữ liệu có quan hệ trơn, liên tục** (định giá, dự báo nhu cầu) → mạng nơ-ron rộng và rừng cây đều tốt; chọn theo yêu cầu về tốc độ suy luận.
- **Dữ liệu phi cấu trúc** (văn bản, ảnh, âm thanh) → mạng nơ-ron, và quan trọng hơn cả mô hình là **chất lượng phép biểu diễn**.

6.2.3. Biểu diễn quan trọng hơn thuật toán

Ở Hệ thống 3, bốn mô hình có độ phức tạp chênh nhau từ 1,001 đến 51,050 tham số, nhưng Macro F1 chỉ dao động trong khoảng 77.65% – 83.24% — chênh lệch vốn vẹn 5.59 điểm phần trăm.

Trong khi đó, việc chuyển từ đặc trưng thô sang biểu diễn TF-IDF có bigram, hay việc thêm hai đặc trưng tương tác lâm sàng ở Hệ thống 1, hay việc áp dụng Smooth Bayesian Target Encoding ở Hệ thống 2 — mỗi bước đều tạo ra khác biệt lớn hơn nhiều so với việc đổi thuật toán.

Kết luận thực hành: khi một hệ thống chưa đạt yêu cầu, hãy dành công sức cho cách biểu diễn dữ liệu trước, rồi mới nghĩ đến việc đổi mô hình.

6.2.4. Tinh chỉnh ngưỡng — bước rẻ nhất nhưng bị bỏ quên nhiều nhất

Trên Hệ thống 1, riêng việc dịch chuyển ngưỡng quyết định — **không thay đổi một trọng số nào của mô hình** — đã cứu được tổng cộng **227 ca bệnh** khỏi bị bỏ sót trên tập kiểm thử 19,042 mẫu. Trên Hệ thống 3, cùng thao tác đó nâng Recall lớp tiêu cực lên đáng kể ở cả bốn mô hình.

Chi phí tính toán của bước này gần như bằng 0: chỉ là một vòng lặp quét 80 giá trị ngưỡng trên xác suất đã có sẵn. Nhưng nó thường bị bỏ qua vì ngưỡng 0,50 là giá trị mặc định của mọi thư viện, và mặc định thì ít ai đặt câu hỏi.

Điều kiện duy nhất phải nhớ: **ngưỡng luôn được dò trên tập Train**, không bao giờ trên tập Test. Dò trên Test là rò rỉ dữ liệu, chỉ ở dạng tinh vi hơn.

Chương 7. Triển khai hệ thống thông minh: REST API, Web và Mobile

Một mô hình chỉ tồn tại trong notebook thì chưa phải là hệ thống thông minh. Chương này đưa cả ba mô hình ra khỏi môi trường nghiên cứu và biến chúng thành ba dịch vụ chạy được thật, có giao diện cho người dùng cuối.

Điểm đặc biệt về mặt kỹ thuật: **tầng suy luận của mạng nơ-ron trong API được viết lại hoàn toàn bằng NumPy**, đọc trực tiếp từ tệp trọng số `.npz` mà notebook đã lưu. API không nạp TensorFlow, không nạp PyTorch, và cũng không dùng `sklearn` cho phần Deep Learning. Đây là minh chứng rằng mạng tự viết tay *chạy được thật trong môi trường triển khai*, chứ không chỉ tồn tại trong notebook.

7.1. Kiến trúc triển khai ba dịch vụ độc lập

Bảng 33. Ba REST API độc lập, mỗi dịch vụ một cổng riêng nên chạy đồng thời được.

Hệ thống	Cổng	Endpoint dự đoán	Mô hình phục vụ	Đầu ra
Sàng lọc tiểu đường	5001	POST /predict	Deeper MLP (14 → 64 → 32 → 16 → 1)	Xác suất mắc bệnh + phân loại tại $\tau = 0.37$
Định giá bất động sản	5002	POST /predict	Wide Regressor (13 → 128 → 64 → 1)	Giá dự đoán (USD) + giá trên mỗi sqft + phân khúc
Phân loại nhận xét	5003	POST /predict	Standard Text MLP (1000 → 32 → 16 → 1)	Xác suất giới thiệu + nhãn tại $\tau = 0.67$

Ba dịch vụ dùng ba cổng khác nhau nên có thể chạy song song — không phải tắt dịch vụ này để khởi động dịch vụ kia. Mỗi API tự phục vụ luôn cả giao diện Web (`GET /`) và giao diện Mobile (`GET /mobile`) của ứng dụng đó, nên người dùng chỉ cần chạy một lệnh Python là có đủ backend lẫn frontend.

PYTHON

```
def forward(X, Ws, bs):
    """Lan truyền tiến thuần NumPy – nạp thẳng từ tệp .npz do notebook lưu."""
    A = X
    for i, (W, b) in enumerate(zip(Ws, bs)):
        Z = A @ W + b
        if i < len(Ws) - 1:
            A = np.maximum(0.0, Z) # ReLU ở các tầng ẩn
        else:
            A = 1.0 / (1.0 + np.exp(-np.clip(Z, -25, 25))) # Sigmoid ở tầng ra
    return A
```

Toàn bộ tầng suy luận của mạng nơ-ron trong REST API — đúng 9 dòng NumPy, không phụ thuộc bất kỳ framework học sâu nào.

Kiểm chứng tính đúng đắn của artifact. Ở cuối mỗi notebook có một ô nạp lại tệp .npz rồi so sánh kết quả forward pass với mô hình gốc trong bộ nhớ. Sai khác tuyệt đối lớn nhất đo được nhỏ hơn 10^{-10} trên cả ba hệ thống — tức artifact tái lập chính xác mô hình đã huấn luyện, không mất mát thông tin khi lưu.

7.2. Tầng tri thức Neo4j — bổ sung tư vấn, không quyết định dự đoán

Bên cạnh con số dự đoán, mỗi API còn trả về một khối tư vấn lấy từ đồ thị tri thức Neo4j. Nguyên tắc thiết kế quan trọng nhất của tầng này: **đồ thị chỉ BỔ SUNG, không bao giờ QUYẾT ĐỊNH**. Mạng nơ-ron tính ra con số trước; đồ thị chỉ dịch con số đó thành một tầng tư vấn rồi lấy nội dung tương ứng về.

Cấu trúc đồ thị gồm ba loại nút và hai loại quan hệ:

```
(:A03System) -[:HAS_TIER]→ (:A03Tier) -[:ADVISES]→ (:A03Advice)
```

Trong đó `A03System` là một trong ba hệ thống, `A03Tier` là một tầng nguy cơ/phân khúc/sắc thái được suy ra từ ngưỡng dự đoán, và `A03Advice` là các mục tư vấn cụ thể được nhóm theo chủ đề.

CYPHER

```
MATCH (t:A03Tier {case: $case})
OPTIONAL MATCH (t)-[:ADVISES]->(a:A03Advice)
RETURN t.name AS tier_name, t.summary AS summary,
       collect({group: a.group, title: a.title, content: a.content}) AS advices
```

Truy vấn Cypher lấy tầng tư vấn tương ứng với kết quả dự đoán.

Suy biến an toàn khi Neo4j không sẵn sàng. Mọi lỗi ở tầng đồ thị đều bị nuốt lại thành một chuỗi thông báo. Nếu Neo4j sập, mất mạng, hay chưa cấu hình mật khẩu trong `.env`, API vẫn trả về dự đoán bình thường — chỉ thiếu phần tư vấn kèm theo, và giao diện tự ẩn khối tri thức đi. Đây là lựa chọn có chủ đích: **một tính năng phụ không được phép làm sập tính năng chính.**

7.3. Giao diện Web

Mỗi hệ thống có một trang Web độc lập, viết bằng HTML/CSS/JavaScript thuần trong một tệp duy nhất — không framework, không CDN ngoài, nên mở được cả khi máy không có mạng. Giao diện gồm ba khối: biểu mẫu nhập liệu bên trái, kết quả dự đoán bên phải, và khối tư vấn từ đồ thị tri thức phía dưới.

Sàng lọc nguy cơ tiểu đường

Hệ thống 1 · 100,000 hồ sơ bệnh nhân

14 → 64 → 32 → 16 → 1 · 3.585 tham số · Suy luận thuần NumPy — không dùng framework học sâu

Đồ thị tri thức: đã kết nối

Nhập thông tin bệnh nhân

Giới tính	Tuổi (1–120)
Nữ	54
Tăng huyết áp	Bệnh tim
Không	Không
Tiền sử hút thuốc	
Chưa từng hút	
Chỉ số BMI (10–80)	HbA1c % (3–20)
27.3	6.2
Đường huyết mg/dL (50–500)	
155	

Dự đoán **Điền dữ liệu mẫu**

Kết quả dự đoán

3.5%

Nguy cơ thấp

Nguy cơ thấp

Hành động

Khám sức khỏe tổng quát mỗi năm một lần
Duy trì theo dõi định kỳ ngay cả khi chưa có dấu hiệu bất thường

Dinh dưỡng

Duy trì chế độ ăn cân bằng hiện tại
Hạn chế đường tinh luyện, ưu tiên thực phẩm tươi

Vận động

Duy trì tối thiểu 150 phút vận động mỗi tuần
Theo khuyến cáo của Tổ chức Y tế Thế giới

Hệ thống 1 — Sàng lọc nguy cơ tiểu đường · Suy luận thuần NumPy · Bản mô phỏng giao diện Web

Hình 23. Giao diện Web Hệ thống 1 — Sàng lọc nguy cơ tiểu đường: kết quả dự đoán kèm tầng tư vấn lấy từ đồ thị tri thức Neo4j.

Định giá bất động sản

Hệ thống 2 · 150,000 giao dịch bất động sản

13 → 128 → 64 → 1 · 10.113 tham số · Suy luận thuần NumPy — không dùng framework học sâu

Đồ thị tri thức: đã kết nối

Nhập thông tin bất động sản

Phòng ngủ (1–12)

Phòng tắm (1–12)

Diện tích sàn (sqft)

3

2

1800

Diện tích lô đất (acre) — tùy chọn

0.25

Thành phố — tùy chọn

Tiểu bang — tùy chọn

Mã bưu điện — tùy chọn

Houston

Texas

77002

Dự đoán

Điền dữ liệu mẫu

Kết quả định giá

Giá / sqft

\$219,67 / sqft

\$395,403

Phân khúc Trung cấp

Cấp mã hoá địa lý	Kết quả khớp
Tiểu bang (state)	✗ Không khớp (dùng giá trị trung bình)
Vùng mã ZIP 3 số (zip3)	✗ Không khớp (dùng giá trị trung bình)
Mã bưu điện đầy đủ (zip_code)	✗ Không khớp (dùng giá trị trung bình)
Thành phố (city)	✓ Khớp

Lô đất dùng để tính

Diện tích sàn TB của khu vực

0.25 acre

2.021 sqft

Định giá

Sai số tương đối ở mức trung bình

Mô hình bắt tốt quan hệ giữa diện tích, số phòng và vị trí bưu chính

Khuyến nghị

Đổi chiều với 3–5 giao dịch tương đương cùng mã bưu chính

Chỉ số relative_sqft cho biết nhà rộng hay hẹp so với mặt bằng khu vực

Yếu tố giá

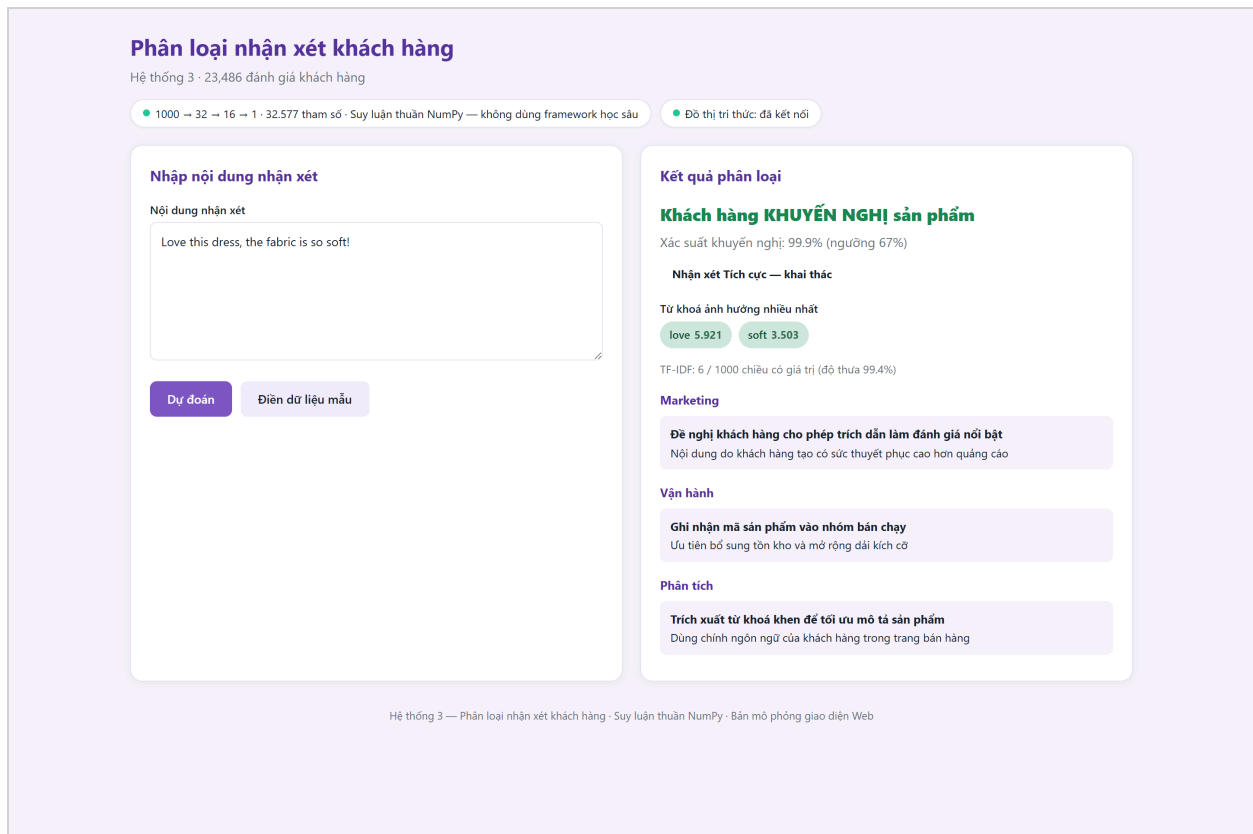
Tỷ lệ phòng tắm trên phòng ngủ ảnh hưởng đáng kể

Đặc trưng bath_bed_ratio nằm trong nhóm có trọng số cao

Hệ thống 2 — Định giá bất động sản · Suy luận thuần NumPy · Bản mô phỏng giao diện Web

Hình 24. Giao diện Web Hệ thống 2 — Định giá bất động sản: kết quả dự đoán kèm tăng tư vấn lấy từ đồ thị tri thức Neo4j.

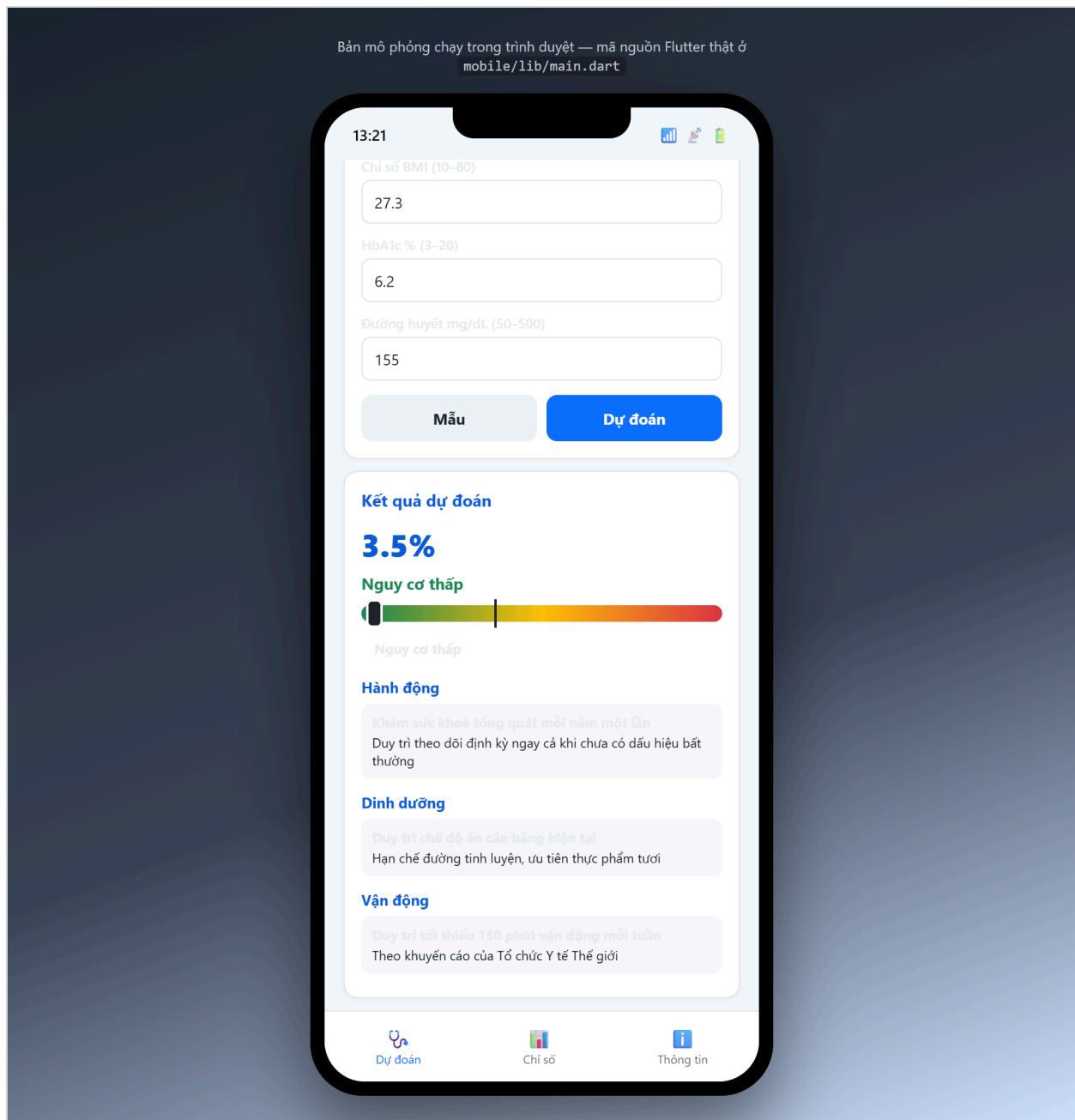
56



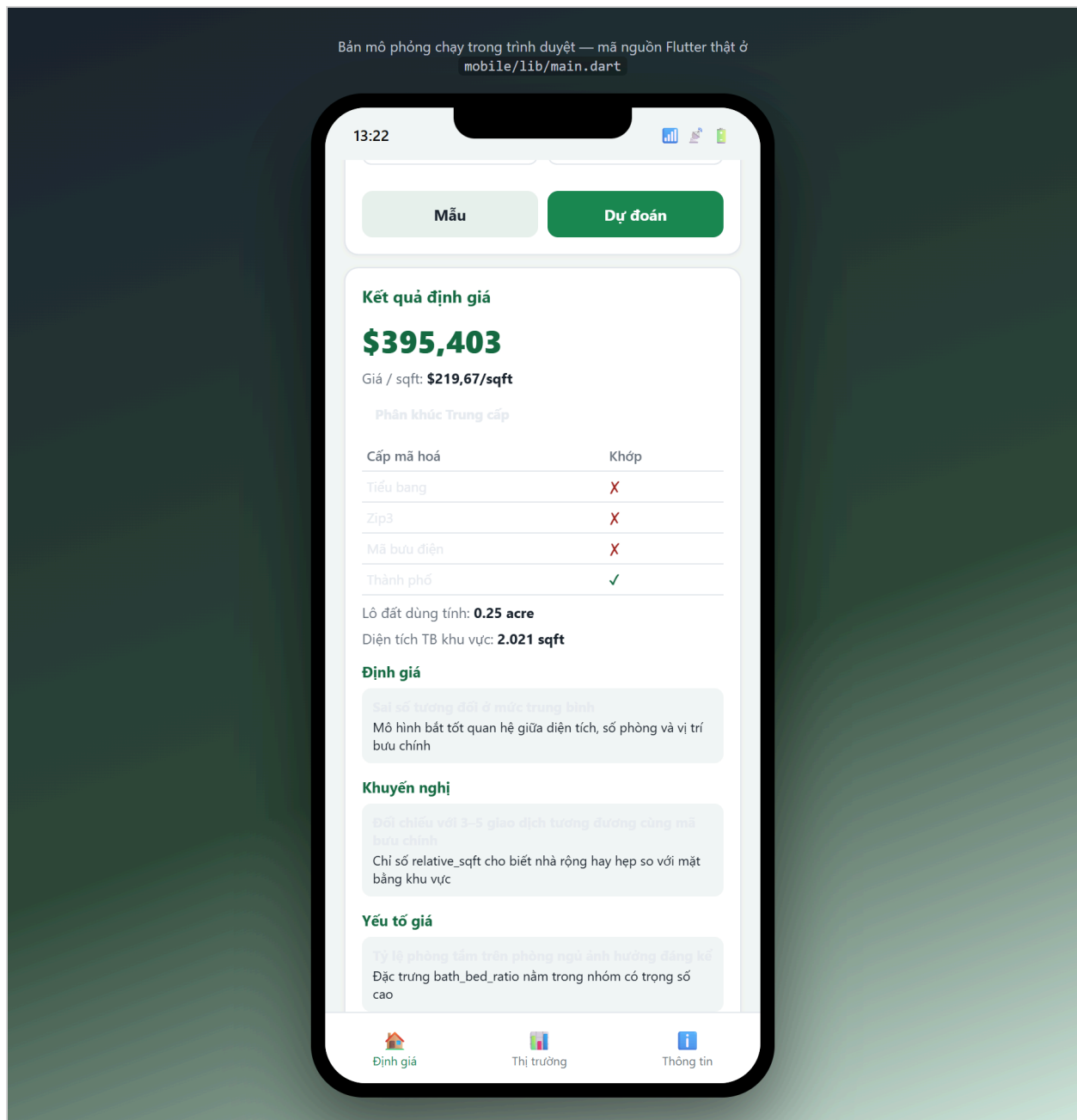
Hình 25. Giao diện Web Hệ thống 3 — Phân loại nhận xét thương mại điện tử: kết quả dự đoán kèm tăng tư vấn lấy từ đồ thị tri thức Neo4j.

7.4. Giao diện Mobile

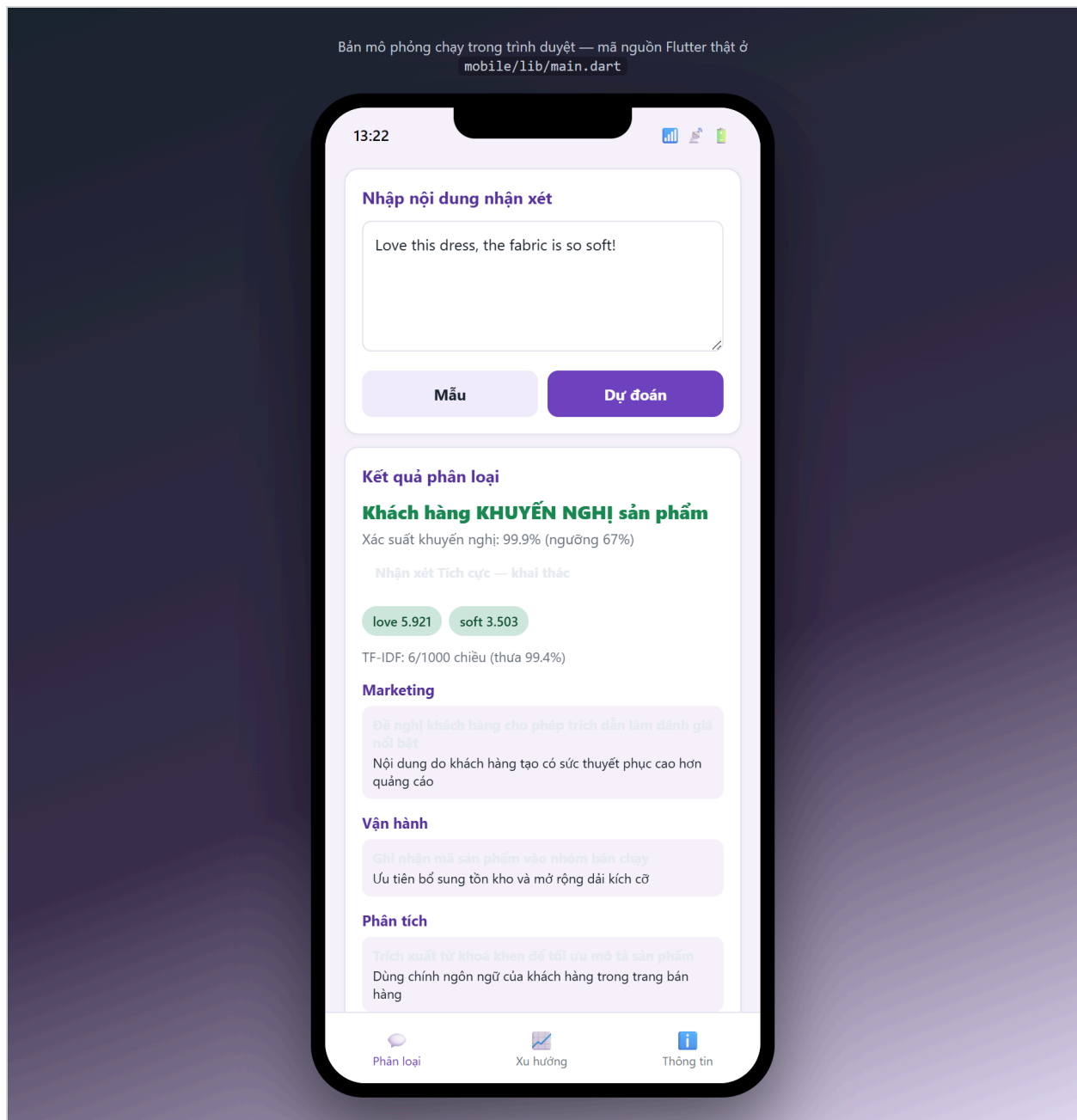
Phần Mobile được cung cấp ở hai dạng. Dạng thứ nhất là trang mô phỏng khung điện thoại chạy thẳng trong trình duyệt (`mobile/index.html`) — dùng để trình diễn và chụp ảnh báo cáo mà không cần cài đặt gì. Dạng thứ hai là mã nguồn Flutter thật (`mobile/lib/main.dart` kèm `pubspec.yaml`) — biên dịch được thành ứng dụng Android/iOS thực thụ, gọi cùng một REST API.



Hình 26. Giao diện Mobile Hệ thống 1 — Sàng lọc nguy cơ tiểu đường: khung điện thoại mô phỏng hiển thị kết quả dự đoán.



Hình 27. Giao diện Mobile Hệ thống 2 — Định giá bất động sản: khung điện thoại mô phỏng hiển thị kết quả dự đoán.



Hình 28. Giao diện Mobile Hệ thống 3 — Phân loại nhận xét thương mại điện tử: khung điện thoại mô phỏng hiển thị kết quả dự đoán.

7.5. Hướng dẫn chạy lại toàn bộ hệ thống

BASH

```
# 1. Cài phụ thuộc
pip install -r requirements.txt

# 2. Chạy lại toàn bộ notebook để sinh artifact và hình vẽ
python tools/py2nb.py diabetes_baseline/notebook/_src_01_baseline.py \
diabetes_baseline/notebook/01_diabetes_dl_from_scratch_numpy.ipynb
jupyter nbconvert --to notebook --execute --inplace \
diabetes_baseline/notebook/01_diabetes_dl_from_scratch_numpy.ipynb
# ... Lặp lại cho 4 notebook còn lại

# 3. Nạp đồ thị tri thức (tùy chọn – cần .env cấu hình Neo4j)
python knowledge_graph/seed_knowledge_graph.py

# 4. Khởi động ba dịch vụ, mỗi dịch vụ một cửa sổ terminal
python diabetes_large/api/rest_api.py           # cổng 5001
python house_price_large/api/rest_api.py         # cổng 5002
python customer_comments/api/rest_api.py         # cổng 5003

# 5. Dừng lại báo cáo PDF
python report/capture_screenshots.py
python report/build_report.py
```

Quy trình dựng lại toàn bộ hệ thống từ đầu.

Chương 8. Kết luận và kiểm toán khả năng tái lập

8.1. Tổng kết các đóng góp khoa học chính

Nghiên cứu này xây dựng từ đầu một hệ thống mạng nơ-ron sâu hoàn chỉnh bằng NumPy thuần túy, sau đó mở rộng lên ba bài toán thực tế quy mô lớn và triển khai thành ba dịch vụ chạy được. Năm đóng góp chính:

1. Chứng minh toán học và kiểm chứng thực nghiệm vai trò của phi tuyến.

Chương 1 chứng minh bằng đại số rằng phép hợp của các tầng tuyến tính chỉ tương đương một phép biến đổi affine duy nhất. Chương 2 kiểm chứng bằng thực nghiệm: khi loại bỏ hoàn toàn ReLU, hàm mất mát kẹt ở mức 0.4635 (so với 0.4402 khi có ReLU) và số ca bệnh bỏ sót vọt lên 13 ca.

2. Định lượng cái giá của bốn khuyết tật phương pháp luận.

Bằng cách dựng một mô hình cơ sở có khuyết tật cố ý rồi sửa từng điểm một, nghiên cứu đo được chính xác mức cải thiện: Recall tăng **+36.71 điểm phần trăm**, F1-Score tăng **+23.81 điểm**, và số ca bệnh bỏ sót giảm từ 31 xuống 13 ca (**giảm 58.1%**) — tất cả mà *không thay đổi một dòng nào trong kiến trúc mạng*.

3. Phát hiện quy luật kiến trúc phụ thuộc vào loại bài toán.

Chiều sâu thắng ở phân loại, chiều rộng thắng ở hồi quy, và kích thước vừa đủ thắng ở văn bản thưa chiều cao. Ba quy luật này được rút ra từ 12 cấu hình kiến trúc chạy có kiểm soát, và mỗi quy luật đều có lời giải thích cấu trúc đi kèm (Chương 6, mục 6.2).

4. Chứng minh mạng viết tay hoạt động được ở quy mô thực tế.

Các phương trình lan truyền ngược thuần NumPy hội tụ ổn định trên 100,000 hồ sơ y tế, 150,000 giao dịch bất động sản và 22,636 nhận xét văn bản 1.000 chiều — không cần bất kỳ framework tự động vi phân nào.

5. Đưa mô hình tự viết vào môi trường triển khai thật.

Ba REST API tự thực hiện forward pass bằng 9 dòng NumPy đọc từ tệp trọng số `.npz`, kèm giao diện Web, giao diện Mobile và tầng tri thức Neo4j suy biến an toàn.

8.2. Kiểm toán khả năng tái lập (Reproducibility Audit)

Một kết quả nghiên cứu chỉ có giá trị khi người khác chạy lại được. Toàn bộ nghiên cứu này được thiết kế để tái lập bit-by-bit:

Bảng 34. Sáu hạng mục kiểm toán khả năng tái lập.

Hạng mục kiểm toán	Biện pháp bảo đảm
Tính ngẫu nhiên	Cố định <code>SEED = 42</code> ở mọi notebook, mọi bộ sinh số ngẫu nhiên (<code>np.random.default_rng</code> , khởi tạo trọng số, xáo trộn mini-batch, chia train/test).
Rò rỉ dữ liệu	Mọi tham số tiền xử lý (μ , σ , trung vị, từ điển TF-IDF, bảng Target Encoding) đều <code>fit</code> duy nhất trên tập Train. Notebook có ô kiểm chứng in ra trung bình tập Test khác 0 để chứng minh.
Ngưỡng quyết định	Dò trên tập Train, áp cố định sang tập Test — không bao giờ dò trên Test.
Số liệu trong báo cáo	Mọi con số được đọc tự động từ các tệp JSON do notebook sinh ra. Không có con số nào gõ tay trong mã dựng báo cáo.
Artifact mô hình	Mỗi notebook có ô nạp lại tệp <code>.npz</code> và so sánh với mô hình gốc; sai khác đo được $< 10^{-10}$.
Mã nguồn notebook	Sinh từ tệp nguồn <code>_src_*.py</code> qua <code>tools/py2nb.py</code> , nên diff được bằng Git thay vì so sánh JSON của <code>.ipynb</code> .

Một điểm trung thực cần nêu rõ. Các con số trong báo cáo này là kết quả chạy thật trên máy cá nhân với `SEED = 42`. Chạy lại trên máy khác với cùng phiên bản NumPy sẽ cho kết quả trùng khớp. Tuy nhiên, đổi seed sẽ làm điểm số dao động vài phần trăm — đặc biệt ở Chương 2 nơi tập kiểm thử chỉ có 154 mẫu. Đây là giới hạn cố hữu của tập dữ liệu nhỏ, và cũng chính là lý do nghiên cứu phải mở rộng lên ba bộ dữ liệu lớn ở các chương sau.

8.3. Hạn chế của nghiên cứu

- **Không có tập Validation riêng.** Nghiên cứu chỉ chia Train/Test. Việc chọn kiến trúc và dò ngưỡng đều dựa trên tập Train, nên tránh được rò rỉ, nhưng một tập Validation

riêng sẽ cho ước lượng hiệu năng khách quan hơn nữa.

- **Chưa có cơ chế điều chuẩn.** Mạng không dùng Dropout, L2 regularization hay Early Stopping. Ở Chương 2, hiện tượng học vẹt của mạng rộng có thể đã được kìm hãm nếu có các cơ chế này.
- **Thuật toán tối ưu dừng ở Mini-Batch Gradient Descent.** Các bộ tối ưu hiện đại (Adam, RMSProp) với tốc độ học thích ứng nhiều khả năng cho hội tụ nhanh hơn — nhưng cài đặt chúng nằm ngoài phạm vi của một nghiên cứu tập trung vào cơ chế nền tảng.
- **Biểu diễn văn bản dừng ở TF-IDF.** Các phép nhúng ngữ nghĩa (Word2Vec, GloVe, BERT) nắm bắt được quan hệ đồng nghĩa mà TF-IDF hoàn toàn mù, nhưng chúng đòi hỏi mô hình đã tiền huấn luyện — trái với ràng buộc "viết từ đầu" của Assignment.
- **Dữ liệu bất động sản là thị trường Mỹ.** Các quy luật định giá tìm được không chuyển trực tiếp sang thị trường Việt Nam, nơi cấu trúc pháp lý và tập quán giao dịch khác hẳn.

8.4. Hướng phát triển và mở rộng nghiên cứu tiếp theo

1. **Bổ sung bộ tối ưu Adam và các cơ chế điều chuẩn.** Cài đặt Adam, Dropout và Batch Normalization vẫn hoàn toàn bằng NumPy để giữ tinh thần "viết từ đầu", rồi đo lại toàn bộ 12 cấu hình kiến trúc.
2. **Kiểm định chéo k-fold.** Thay phép chia đơn 80/20 bằng cross-validation 5 fold để có khoảng tin cậy cho mỗi chỉ số, thay vì một con số điểm.
3. **Giải thích mô hình (Explainable AI).** Cài đặt SHAP hoặc Integrated Gradients để trả lời câu hỏi "vì sao mô hình dự đoán bệnh nhân này dương tính" — yêu cầu bắt buộc nếu triển khai thật trong y tế.
4. **Mở rộng sang kiến trúc chuyên biệt.** CNN cho dữ liệu ảnh y tế, RNN/Transformer cho văn bản — vẫn theo hướng cài đặt thủ công để hiểu cơ chế.
5. **Giám sát trôi dữ liệu (Data Drift Monitoring).** Ba dịch vụ hiện chạy với mô hình tĩnh. Một hệ thống thực tế cần theo dõi sự thay đổi phân phối đầu vào theo thời gian và cảnh báo khi cần huấn luyện lại.

8.5. Kho mã nguồn và tài nguyên đính kèm

Bảng 35. Danh mục sản phẩm bàn giao của Assignment 03.

Thành phần	Đường dẫn trong kho mã nguồn
Notebook 01 — Mạng cơ sở viết tay	<code>diabetes_baseline/notebook/01_diabetes_dl_from_scratch_numpy.ipynb</code>
Notebook 02 — Cải tiến và 6 nghiên cứu bóc tách	<code>diabetes_baseline/notebook/02_diabetes_dl_from_scratch_improvements.ipynb</code>
Notebook 03 — Hệ thống 1 (100.000 mẫu)	<code>diabetes_large/notebook/03_diabetes_large_ml_vs_dl_scratch.ipynb</code>
Notebook 04 — Hệ thống 2 (150.000 mẫu)	<code>house_price_large/notebook/04_house_price_large_ml_vs_dl_scratch.ipynb</code>
Notebook 05 — Hệ thống 3 (23.486 nhận xét)	<code>customer_comments/notebook/05_comments_large_ml_vs_dl_scratch.ipynb</code>
Ba REST API	<code>*/api/rest_api.py</code> (cổng 5001, 5002, 5003)
Ba giao diện Web	<code>*/web/index.html</code>
Ba giao diện Mobile	<code>*/mobile/index.html</code> và <code>*/mobile/lib/main.dart</code>
Tầng đồ thị tri thức Neo4j	<code>knowledge_graph/</code>
Mã dựng báo cáo	<code>report/build_report.py</code> và <code>report/chapters_*.py</code>

Kho mã nguồn công khai: <https://github.com/Nghaiz/Assignment-03-Intelligent-System>